

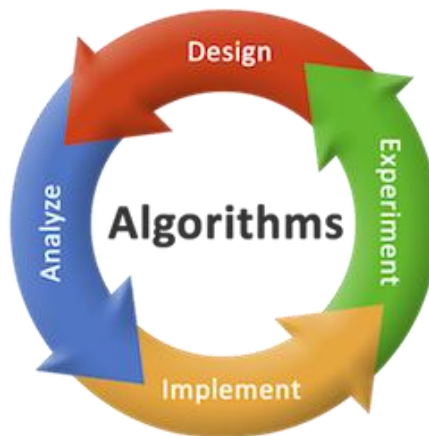
South Valley University

جامعة جنوب الوادي



Computer and Artificial Intelligence Program

Algorithms Analysis and Design



Level Three

Code :CS311

Introduction

In today's data-driven world, algorithms are the invisible engines behind the technologies we use every day—from search engines and social media feeds to medical diagnostics and financial systems. They are not just lines of code; they are logical blueprints that solve complex problems, automate decisions, and optimize performance across nearly every industry.

This document, “**Algorithm Fundamentals**,” provides a comprehensive foundation in algorithmic thinking—a critical skill for every computer science student. Through a blend of theory, real-world applications, and structured design techniques, this material empowers students to understand not only how algorithms work, but *why* they work, *when* to use them, and *how* to evaluate their effectiveness.

Covering essential topics such as algorithm characteristics, representation methods (natural language, pseudocode, flowcharts), time and space complexity analysis, sorting and searching techniques, and advanced design strategies like divide-and-conquer, greedy methods, and dynamic programming, this resource bridges the gap between abstract theory and practical implementation.

Whether you're aspiring to become a software engineer, data scientist, or researcher, mastering these fundamental concepts will prepare you to design intelligent, efficient, and scalable solutions to real-world challenges.

Course Objectives

- To introduce fundamental techniques for designing efficient algorithms and analyzing their complexity.
- To develop the ability to evaluate time and space requirements of algorithms using mathematical models.
- To familiarize students with classical algorithm design paradigms such as divide-and-conquer, dynamic programming, greedy methods, and backtracking.

- To expose students to advanced topics like graph algorithms, NP-completeness, and approximation algorithms.
- To enhance problem-solving skills through rigorous exercises and case studies relevant to computer science and AI applications.

Course Competencies

By the end of the course, students will be able to:

- Understand and apply algorithmic complexity analysis using Big-O, Big-Theta, and Big-Omega notations.
- Design algorithms using standard paradigms (divide-and-conquer, greedy, dynamic programming, backtracking).
- Analyze and compare multiple algorithms to choose the most efficient solution for a given problem.
- Implement algorithms in a programming language (e.g., C++/Python) with proper documentation and testing.
- Evaluate computational problems to determine whether they are tractable, intractable, or NP-complete.
- Apply algorithmic techniques to real-world scenarios in data processing, networks, artificial intelligence, and optimization.
- Demonstrate teamwork and communication skills by presenting algorithmic solutions and collaborating on problem sets.

Content

Introduction	2
CHAPTER 1 : Introduction	8
1.1 What is an Algorithm?	8
1.2 Characteristics of Algorithms	9
1.3 Ways to Write and Represent Algorithms	10
1.4 Algorithm vs. Program	13
1.5 Core Algorithm Components	15
1.7 Algorithm Examples	17
Questions	20
CHAPTER 2: ANALYSIS OF ALGORITHMS	21
Introduction	21
2.1 Why Analysis of Algorithms?	21
Goal of Analysis of Algorithms?	21
How to Compare Algorithms?	22
Total Cost = cost_of_car + cost_of_cycle	22
Total Cost $\approx$ cost_of_car (approximation)	22
2.2 Types of Analysis	24
1. Worst case	24
2. Best case	24
3. Average case	24
1. Big-O Notation	25
2. Omega Notation	28
3. Theta- Notation	29
Questions	31
CHAPTER 3: Mathematical Analysis of Nonrecursive Algorithms and Recursive Algorithms	33
3.1 Mathematical Analysis of Nonrecursive Algorithms	33
Purpose of the Analysis	33

Why Use Mathematical Analysis?	33
Analysis Steps	40
Time Complexity	40
Analysis Steps	41
3.2 Mathematical Analysis of Recursive Algorithms	43
Purpose of the Analysis	43
Why Analyze Recursive Algorithms Separately?	43
1. Input size:	44
2. Basic Operation:	44
Problem Statement	46
Analysis	Error! Bookmark not defined.
Step 1: Set up the Recurrence Relation	Error! Bookmark not defined.
Step 2: Base Case (Initial Condition)	Error! Bookmark not defined.
Solving the Recurrence	Error! Bookmark not defined.
Questions	51
CHAPTER 4: Sorting Algorithms	54
4.1-Bubble Sort	54
What is Bubble Sort?	
How Does It Work?	54
Time and Space Complexity	56
Pseudocode	57
Bubble Sort Pseudocode Analysis	57
Time Complexity Analysis	58
Worst-case Time Complexity:	58
Best-case Time Complexity:	59
4.2 Merge Sort	59
What is Merge Sort?	59
	60
	61

How Merge Sort Works – Step-by-Step	61
Merge Sort Pseudocode Explanation	62
What is MergeOrdered?	63
Time Complexity Analysis	64
Recursive Depth:	64
Work Per Level:	64
Total Time: $T(n)=O(n\log n)$	64
Time Complexity	64
Space Complexity	65
4.3 Insertion Sort Algorithm	65
Algorithm: Insertionsort(list)	66
Time Complexity Analysis:	68
Best Case: Already Sorted List	68
Average Case: Random Order	68
Worst Case: Reverse Sorted List	69
4.4 Quick Sort Algorithm	70
Time Complexity of Quick Sort	73
Questions	76
CHAPTER 5: SEARCHING ALGORITHMS	77
Introduction to Searching Algorithms	77
Why Searching is Important:	78
5.1 Types of Searching Algorithms:	78
1- Sequential Search Algorithm	78
2- Probability Search Algorithm	80
3- Binary Search Algorithm	85
What is Binary Search?	85
Key Conditions:	85
Algorithm Steps (Pseudocode)	86
Explanation of Steps:	86

Questions	88
CHAPTER 6 :Divide-and-Conquer	90
Why Use Divide-and-Conquer?	90
6.1 Divide-and-Conquer Paradigm	90
Example: Merge Sort	91
Recurrences	91
6.2 Methods to Solve Recurrences	91
1. Substitution Method	91
2. Recursion Tree Method	92
3. Master Theorem	92
Maximum-Subarray Problem: Overview	92
Brute-force Approach	93
Efficient Solution Using Divide-and-Conquer	93
FIND-MAX-CROSSING-SUBARRAY	94
Time Complexity Analysis:	94
Strassen's Algorithm for Matrix Multiplication (Divide-and-Conquer)	95
1. Substitution Method for Solving Recurrences	97
2. Recursion Tree Method	100
QUESTIONS	107
CHAPTER 7 : Design strategies	108
7.1 Greedy Algorithms	109
7.3 Dynamic Programming (DP)	113
Greedy Algorithms	121

CHAPTER 1 : Introduction To Algorithms

Have you ever wondered how Google finds the best web pages for your search in a fraction of a second?

Or how Netflix knows exactly what movie to recommend next?

How your GPS figures out the fastest route through a maze of city streets?

Behind all these modern marvels lie algorithms — step-by-step methods that solve problems, make decisions, and automate tasks at an unimaginable scale.

Why study algorithms?

Algorithms are everywhere.

They power search engines, guide robots, protect our data, and even help doctors diagnose diseases.

They teach us how to think.

Studying algorithms is not just about programming — it's about learning how to break down complex problems, recognize patterns, and build efficient solutions.

They unlock performance.

A clever algorithm can solve in seconds what a naive approach might take years. It's the ultimate example of “work smarter, not harder.”

What will you learn in this course?

How to design algorithms for a wide range of problems.

How to analyze their efficiency, so you know which solution is best.

Classic techniques like divide and conquer, greedy algorithms, and dynamic programming.

Real-world applications that impact technology, business, and science.

✦ By the end of this course, you won't just be writing code — you'll be crafting solutions that are elegant, powerful, and truly intelligent.

1.1 What is an Algorithm?

An algorithm is a well-defined sequence of unambiguous instructions designed to solve a specific problem.

It takes some input, processes it through a finite number of steps, and produces the desired output.

Definition:

An algorithm is a finite sequence of clear, step-by-step instructions, each of which can be carried out in a finite amount of time, to solve a given problem for all valid inputs.

The concept of algorithms is fundamental in computer science and predates computers themselves.

Historically, even before electronic computers existed, people (called “computers”) followed algorithms to perform calculations.

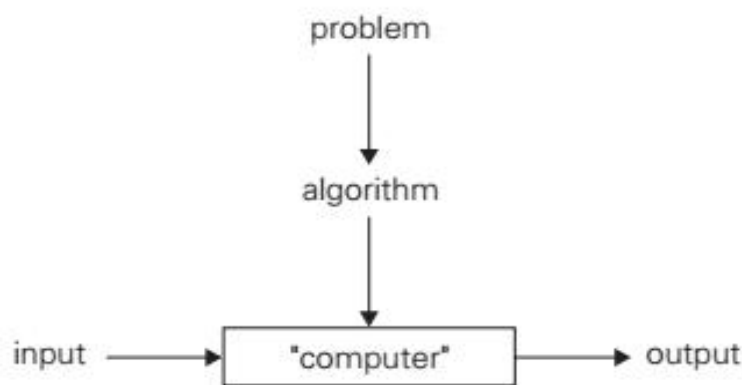


Figure1.An algorithm

1.2 Characteristics of Algorithms

For any process to qualify as an algorithm, it must have the following essential properties:

✓ **Input:**

The algorithm receives zero or more inputs from a specified set.

✓ **Output:**

It produces at least one output (result).

✓ **Definiteness:**

Each step must be precisely and clearly defined. There should be no ambiguity in what needs to be done.

✓ **Finiteness:**

The algorithm must terminate after a finite number of steps. It should not go into an infinite loop.

✓ **Effectiveness:**

All operations must be basic enough to be carried out exactly and within a finite amount of time, in principle even by a person with paper and pencil.

1.3 Ways to Write and Represent Algorithms

There are several standard ways to write or represent algorithms. The choice depends on the audience, purpose, and the level of detail needed.

1- Natural Language Description

This involves writing the algorithm as simple, step-by-step instructions in plain English (or any language).

✓ **Advantages:** Easy for beginners to understand.

✓ **Disadvantages:** Can be ambiguous or too informal for precise communication.

Example:

1. Read two numbers.
2. Add them.
3. Print the result.

2- Pseudocode

Pseudocode uses a structured, programming-like style without worrying about exact syntax of a programming language. It focuses on the logic.

✓ **Advantages:** Easy to read and write; abstracts away programming details.

✓ **Disadvantages:** Needs to be later translated to real code.

Example:

Input A, B
Sum $\leftarrow$ A + B
Output Sum

3- Flowcharts

Flowcharts are graphical diagrams that show the flow of control using standard shapes:

- ✓ Ovals for Start/End,
- ✓ Rectangles for Processes,
- ✓ Diamonds for Decisions,
- ✓ Arrows for flow direction.
- ✓ **Advantages:** Very intuitive; helps visualize the process.
- ✓ **Disadvantages:** Can get messy for complex algorithms.

Example:

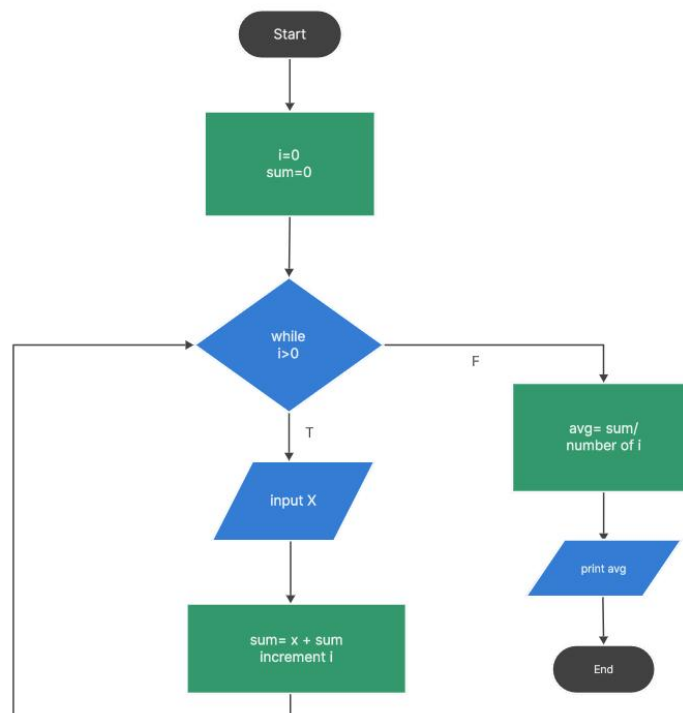
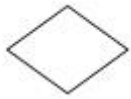


Figure 2 .Flowcharts

Commonly Used Symbols in Detailed Flowcharts

-  Rectangle - flowchart process step One step in the process. The step is written inside the box. Usually, only one arrow goes out of the box.
-  arrow - flowchart flow direction Direction of flow from one step or decision to another.



Decision based on a question. The question is written in the diamond. More than one arrow goes out of the diamond, each one showing the direction the process takes for a given answer to the question. (Often the answers are "yes" and "no.")



Delay or wait



Link to another page or another flowchart. The same symbol on the other page indicates that the flow continues there.



input or output



Document



Alternate symbols for start and end points

4- Programming Code

Writing the algorithm directly in a programming language like Python, C++, or Java.

✓ **Advantages:** Can be executed immediately by a computer.

✓ **Disadvantages:** Less abstract; ties the algorithm to specific syntax and implementation details.

Example in Python:

```
python

def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Best Practices for Students

- 1- **Start Simple:** Use pseudocode for design → code for implementation.
- 2- **Validate with Math:** Prove correctness/complexity formally.
- 3- **Visualize Hard Parts:** Draw flowcharts for recursion or tables for DP.
- 4- **Target Your Audience:**
 - Exams/Textbooks: Pseudocode
 - Developers: Executable Code
 - Researchers: Math + Pseudocode

5. Use Tools:

- Flowcharts: Diagrams.net, Mermaid
- Pseudocode → Code: LeetCode Playground
- Visualization: VisuAlgo, Algorithm Visualizer

1.4 Algorithm vs. Program

An algorithm is a conceptual, step-by-step procedure to solve a problem.

It is language-independent — it does not depend on any specific programming language or machine.

✓ Example:

“To find the largest number in a list, go through each item and keep track of the largest one found so far.”

An algorithm only describes what needs to be done, not how to do it in a particular language.

A program is a specific implementation of an algorithm written in a programming language (like Python, C++, or Java), so that a computer can execute it.

✓ **Example:**

The same algorithm to find the largest number can be written in different programming languages:

In C++:

```
cpp

int max_num = numbers[0];
for (int num : numbers) {
    if (num > max_num) {
        max_num = num;
    }
}
cout << max_num;
```

Feature	Algorithm	Program
Definition	A logical step-by-step solution	A code implementation of an algorithm
Language	Independent of any language	Written in a programming language
Execution	Cannot be executed by a computer	Can be executed by a computer
Focus	Focuses on logic & steps	Focuses on syntax & implementation

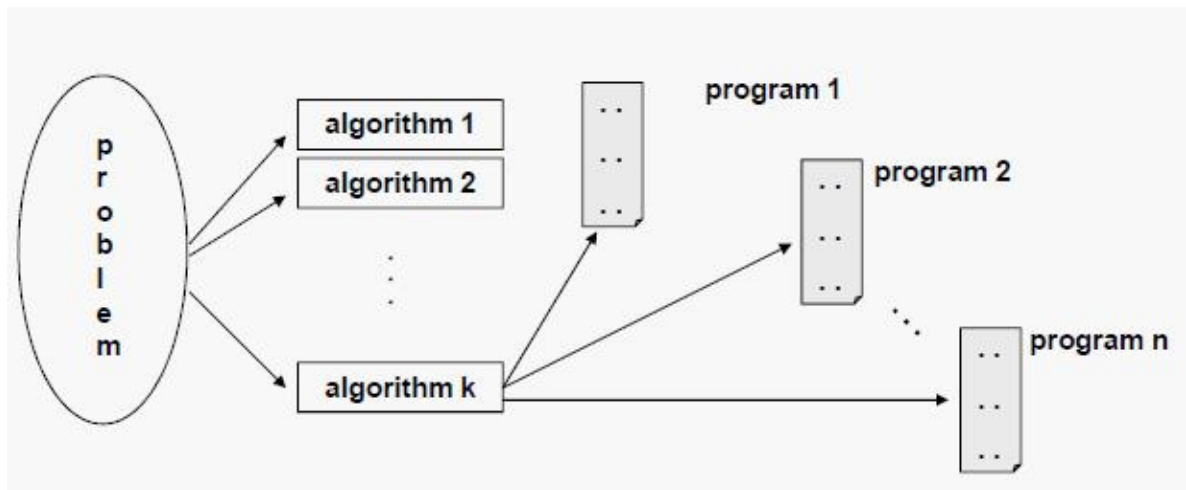


Figure 3. Algorithm vs. Program

1.5 Core Algorithm Components

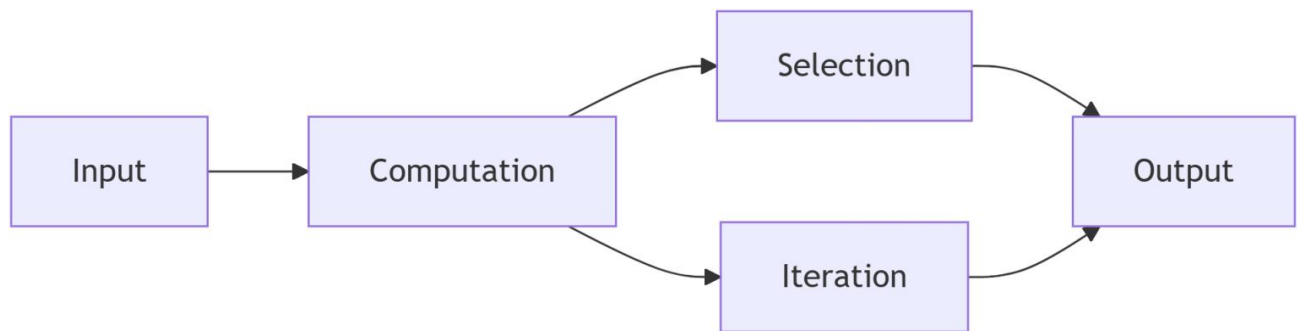


Figure 4. Algorithm Components

- **Input:** Acquire data (e.g., READ grade)
- **Computation:** Arithmetic/logical operations
- **Selection:** Decision-making (IF-THEN-ELSE)
- **Iteration:** Repetition (WHILE, FOR)
- **Output:** Report results (PRINT result)

1.6 Algorithm Design Process

Designing an algorithm is a systematic process that involves several stages. It ensures that the solution is correct, efficient, and practical for implementation.

➤ Steps in the Algorithm Design Process

1- Understanding the Problem

- Carefully read and analyze the problem statement.
- Identify the inputs, required outputs, and any constraints.

- Ask questions like:
 - What exactly is being asked?
 - What are the edge cases?
 - Are there performance constraints?

2- Developing a Strategy

- Decide how to approach the problem.
- Choose an overall technique:
 - Greedy algorithms
 - Divide and conquer
 - Dynamic programming
 - Brute force
- Sometimes this means considering multiple approaches and comparing them.

3- Designing the Algorithm

- Write a high-level sequence of steps that solves the problem.
- Represent it in pseudocode or a flowchart.
- Ensure each step is clear and unambiguous.

4- Proving Correctness

- Show that the algorithm works for all valid inputs.
- Use logical reasoning or formal proofs if necessary.
- This helps avoid hidden logical errors.

5- Analyzing Efficiency

- Determine the time complexity (how running time grows with input size).
- Determine the space complexity (how much memory it uses).
- Helps decide if the algorithm is practical.

6- Implementing the Algorithm

- Translate the pseudocode into an actual programming language (Python, Java, C++, etc).
- Pay attention to data structures and syntax.

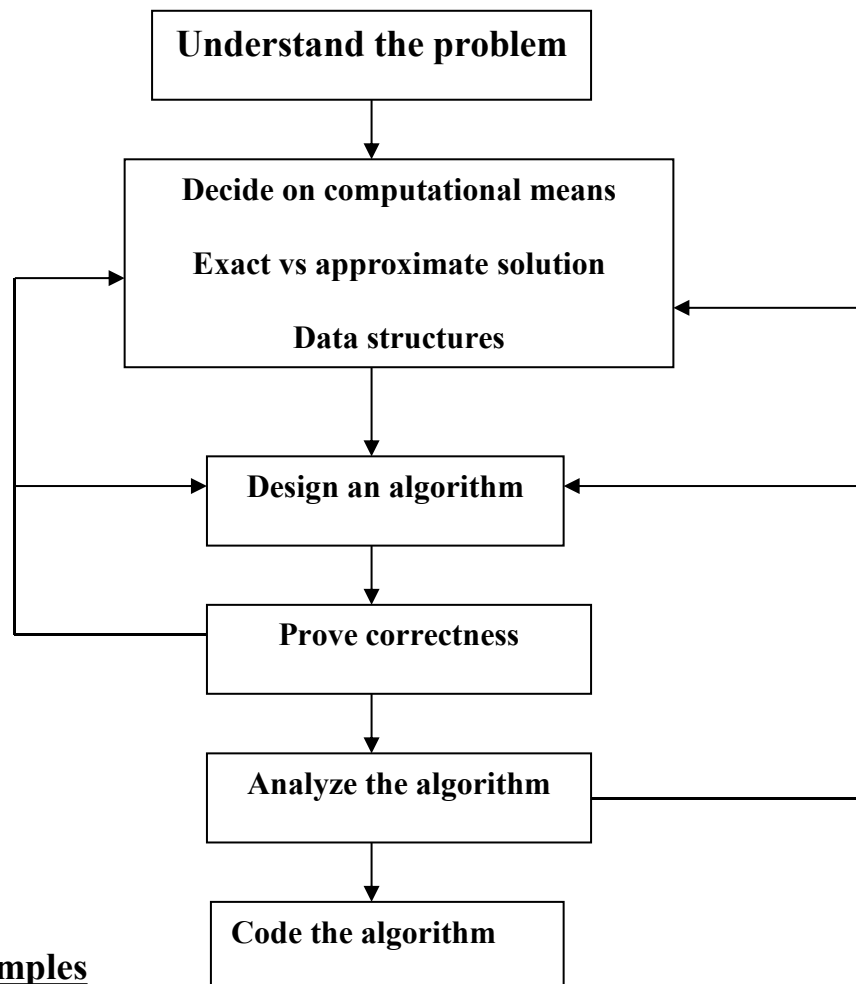
7- Testing and Debugging

- Test the program with typical cases, edge cases, and large inputs.
- Fix any bugs or inefficiencies.

Iteration and Improvement

● Often, after implementation and testing, improvements are needed:

- Optimize performance.
- Handle additional edge cases.
- Make the code cleaner or more general.



1.7 Algorithm Examples

1- Finding the Maximum Number in a List

Goal: Given a list of numbers, find the largest one.

Algorithm:

1. Set $\text{max} \leftarrow$ first element in the list
2. For each element x in the list:

if $x > \text{max}$ then

set $\text{max} \leftarrow x$

3. Return max

✓ Explanation:

We start by assuming the first number is the largest, then compare each number in the list to update the maximum if we find a bigger one.

2- Calculating the Factorial of a Number

Goal: Given a positive integer n , compute $n! = n \times (n-1) \times \dots \times 2 \times 1$.

Algorithm:

1. Set $\text{result} \leftarrow 1$

2. For i from 2 to n :

set $\text{result} \leftarrow \text{result} \times i$

3. Return result

✓ Example:

If $n = 5$, then $\text{result} = 1 \times 2 \times 3 \times 4 \times 5 = 120$.

3- Linear Search

Goal: Find a target value in a list.

Algorithm:

1. For each element x in the list:

If x equals the target, then

→ return "Found".

2. If we reach the end of the list without finding the target,

→ return "Not found".

Explanation:

We check each element one by one until we find the target (or confirm it's not in the list).

Euclidean Algorithm for GCD

Goal: Compute the greatest common divisor (GCD) of two integers a and b .

Algorithm:

While $b \neq 0$:

Set temp = b.
Set b = a mod b.
Set a = temp.
When b becomes 0, a is the GCD.
Return a.

Explanation:

The Euclidean algorithm works by repeatedly replacing the larger number with the remainder of dividing the larger number by the smaller one. This process continues until the remainder becomes zero. The last non-zero remainder is the GCD.

Algorithm:

1. While $b \neq 0$ do:

temp $\leftarrow$ b

b $\leftarrow$ a mod b

a $\leftarrow$ temp

2. Return a

✓ **Example:**

For (48,18):

$48 \bmod 18 = 12 \rightarrow (a=18, b=12)$

$18 \bmod 12 = 6 \rightarrow (a=12, b=6)$

$12 \bmod 6 = 0 \rightarrow (a=6, b=0)$

So GCD = 6.

Questions

1. Describe the algorithm used by your favorite ATM machine in dispensing cash. (You may give your description in either English or pseudocode, whichever you find more convenient.)
2. Consider the following algorithm for finding the distance between the two closest elements in an array of numbers.

ALGORITHM *MinDistance*($A[0..n - 1]$)

 //Input: Array $A[0..n - 1]$ of numbers

 //Output: Minimum distance between two of its elements

$dmin \leftarrow \infty$

for $i \leftarrow 0$ **to** $n - 1$ **do**

for $j \leftarrow 0$ **to** $n - 1$ **do**

if $i \neq j$ **and** $|A[i] - A[j]| < dmin$

$dmin \leftarrow |A[i] - A[j]|$

return $dmin$

- What is the input and output of the MinDistance algorithm?
- Why does the algorithm check $i \neq j$ inside the inner loop?
- What does the expression $|A[i] - A[j]|$ compute in the context of this algorithm?
- Manually trace the algorithm on the array $A = [4, 9, 1, 32]$.
- What will the algorithm return if the array contains all identical elements, such as $A = [7, 7, 7, 7]$?
- What happens if the array has fewer than two elements? Is the algorithm safe in this case?

3- Describe the standard algorithm for finding the binary representation of a positive decimal integer

- a. in English.
- b. in pseudocode.

CHAPTER 2: ANALYSIS OF ALGORITHMS

Introduction

The objective of this chapter is to explain the importance of analysis of algorithms, their notations, relationships and solving as many problems as possible. We first concentrate on understanding the importance of analysis and then slowly move towards analyzing the algorithms with different notations and finally, the problems. After completion of this chapter you should be able to find the complexity of any given algorithm (especially recursive functions).

2.1 Why Analysis of Algorithms?

If we want to go from city “ ” to city “ ”. There can be many ways of doing this: by flight, by bus, by train and also by cycle. Depending on the availability and convenience we choose the one which suits us. Similarly, in computer science there can be multiple algorithms exist for solving the same problem (for example, sorting problem has lot of algorithms like insertion sort, selection sort, quick sort and many more). Algorithm analysis helps us determining which of them is efficient in terms of time and space consumed.

Goal of Analysis of Algorithms?

The goal of **Analysis of Algorithms** is to compare algorithms (or solutions) mainly in terms of running time but also in terms of other factors (e.g., memory, developer's effort etc.)

What is Running Time Analysis?

It is the process of determining how processing time increases as the size of the problem (input size) increases. Input size is number of elements in the input and depending on the problem type the input may be of different types. In general, we encounter the following types of inputs.

- Size of an array

- Polynomial degree
- Number of elements in a matrix
- Number of bits in binary representation of the input
- Vertices and edges in a graph

How to Compare Algorithms?

To compare algorithms, we need objective measures.

- Execution times?

Not reliable, because they depend on the specific machine (CPU, RAM, etc.).

- Number of statements executed?

Also not reliable, because it depends on the programming language and the style of the programmer.

✓ Ideal solution:

Express the running time as a function of input size n , written as $f(n)$.

By comparing these functions, we can see which algorithm is more efficient.

This comparison does not depend on hardware or programming style, making it fair and universal.

What is Rate of Growth?

The rate at which the running time increases as a function of input is called Rate of growth. Let us assume that you went to a shop for buying a car and a cycle. If your friend sees you there and asks what you are buying then in general we say buying a car. This is because cost of car is too big compared to cost of cycle (approximating the cost of cycle to cost of car).

$$\text{Total Cost} = \text{cost_of_car} + \text{cost_of_cycle}$$

$$\text{Total Cost} \approx \text{cost_of_car} \text{ (approximation)}$$

For the above example, we can represent the cost of car and cost of cycle in terms of function and for a given function we ignore the low order terms that are relatively insignificant (for large value of input size, n). As an example in the below case n^4 , $2n^2$, $100n$, and 500 are the individual costs of some function and we approximate it to n^4 . Since, n^4 is the highest rate of growth.

$$n^4 + 2n^2 + 100n + 500 \approx n^4$$

Commonly used Rate of Growths

Below diagram shows the relationship between different rates of growth.

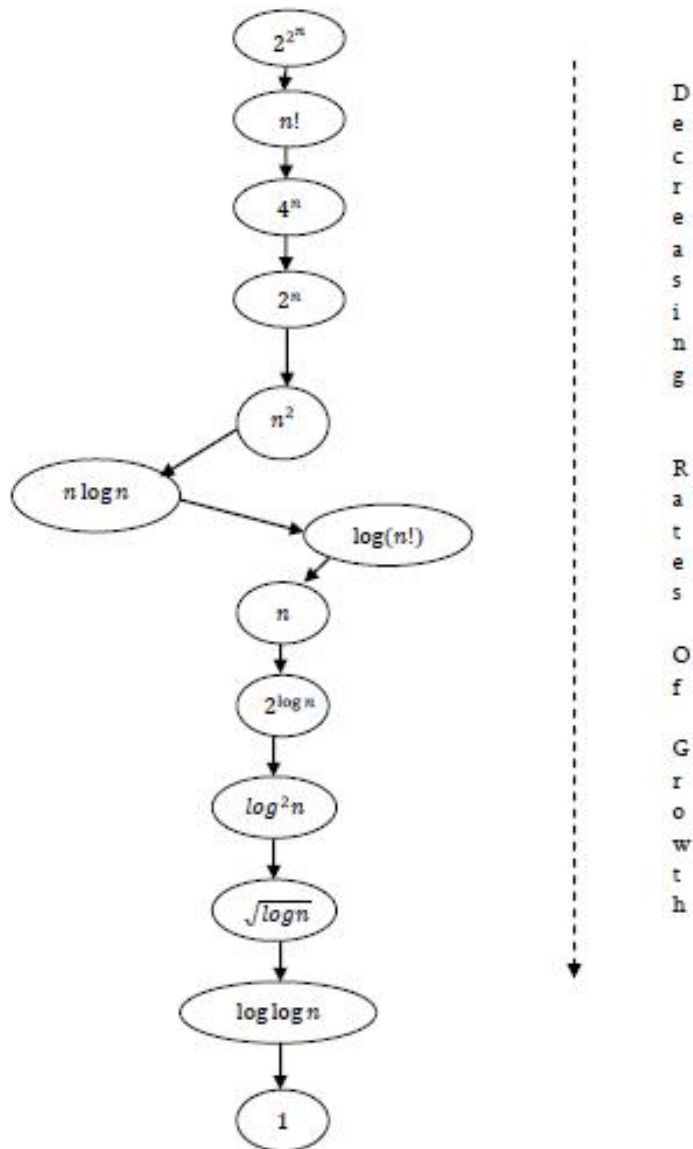


Figure 1. the relationship between different rates of growth

Below is the list of rate of growths which come across in remaining chapters.

Table 1. rate of growths

Time complexity	Name	Example
1	Constant	Adding an element to the front of a linked list
$\log n$	Logarithmic	Finding an element in a sorted array
n	Linear	Finding an element in an unsorted array
$n \log n$	Linear Logarithmic	Sorting n items by 'divide-and-conquer'-Mergesort
n^2	Quadratic	Shortest path between two nodes in a graph
n^3	Cubic	Matrix Multiplication
2^n	Exponential	The Towers of Hanoi problem

2.2 Types of Analysis

If we have an algorithm for a problem and want to know on what inputs the algorithm is taking less time

(performing well) and on what inputs the algorithm is taking huge time.

We have already seen that an algorithm can be represented in the form of an expression. That means we

represent the algorithm with multiple expressions: one for case where it is taking the less time and other for case where it is taking the more time. In general the first case is called the best case and second case is

called the worst case for the algorithm.

To analyze an algorithm we need some kind of syntax and that forms the base for asymptotic

analysis/notation. There are three types of analysis:

1. Worst case

- o Defines the input for which the algorithm takes huge time.
- o Input is the one for which the algorithm runs the slower.

2. Best case

- o Defines the input for which the algorithm takes lowest time.
- o Input is the one for which the algorithm runs the fastest.

3. Average case

- o Provides a prediction about the running time of the algorithm

- o Assumes that the input is random

$$\text{Lower Bound} \leq \text{Average Time} \leq \text{Upper Bound}$$

For a given algorithm, we can represent best case, worst case, and average case analysis in the form of expressions. As an example, let $f(n)$ be the function which represents the given algorithm.

$$\begin{aligned} f(n) &= n^2 + 500, \text{ for worst case} \\ f(n) &= n + 100n + 500, \text{ for best case} \end{aligned}$$

Similarly, for average case too. The expression defines the inputs with which the algorithm takes the

average running time (or memory).

Asymptotic Notation?

Having the expressions for best case, average case and worst case, for all the three cases we need to identify

the upper bound, lower bounds. In order to represent these upper bound and lower bounds we need some

syntax and that is the subject of following discussion. Let us assume that the given algorithm is represented in the form of function $f(n)$.

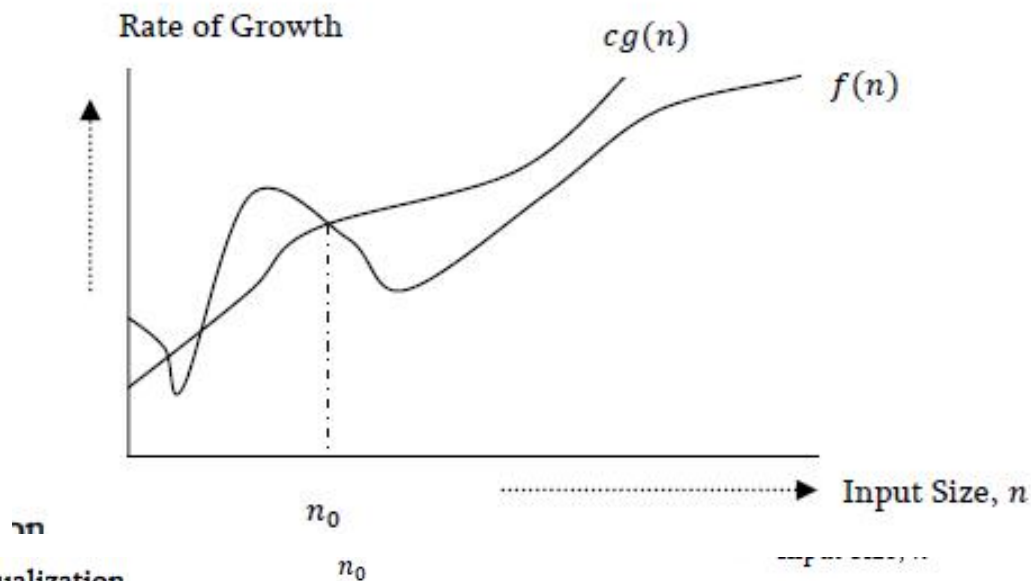
1. Big-O Notation

This notation gives the *tight* upper bound of the given function. Generally we represent it as $f(n) = O(g(n))$. That means, at larger values of n , the upper bound of $f(n)$ is $g(n)$.

For example, if $f(n) = n^4 + 100n^2 + 10n + 50$ is the given algorithm, then n^4 is $g(n)$. That means $g(n)$ gives the maximum rate of growth for $f(n)$ at larger values of n .

Let us see the O –notation with little more detail. O –notation defined as $O(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$. $g(n)$ is an asymptotic tight upper bound for $f(n)$. Our objective is to give smallest rate of growth $g(n)$ which is greater than or equal to given algorithms rate of growth $f(n)$.

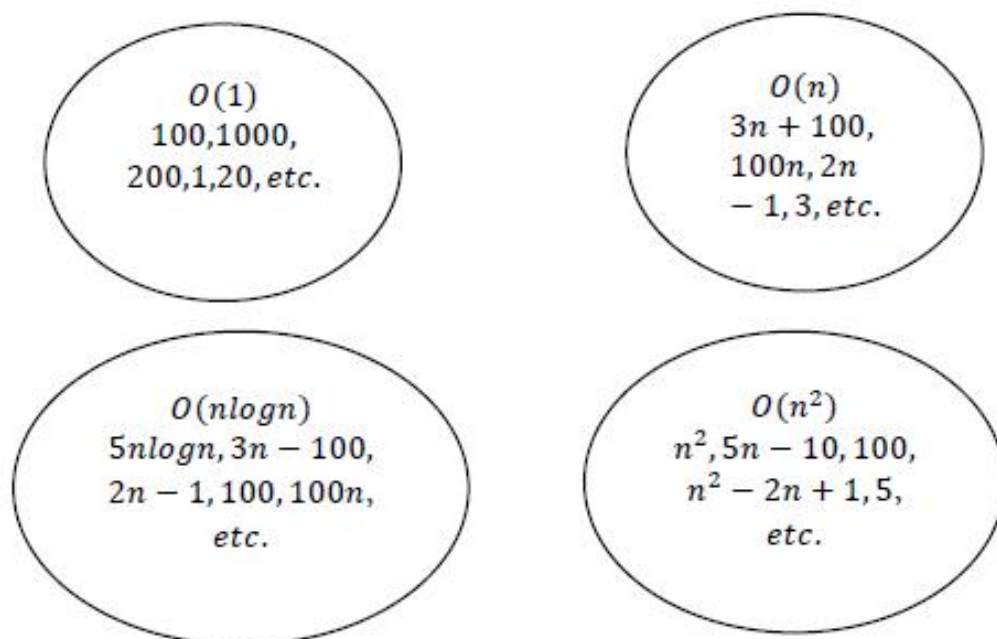
In general, we discard lower values of n . That means the rate of growth at lower values of n is not important. In the below figure, n_0 is the point from which we need to consider the rate of growths for a given algorithm. Below n_0 the rate of growths could be different.



Big-O Visualization

$O(g(n))$ is the set of functions with smaller or same order of growth as $g(n)$. For example, $O(n^2)$ includes $O(1)$, $O(n)$, $O(n \log n)$ etc..

Note: Analyze the algorithms at larger values of n only. What this means is, below n_0 we do not care for rate of growth.



Big-O Examples

Example-1 Find upper bound for $f(n) = 3n + 8$

Solution: $3n + 8 \leq 4n$, for all $n \geq 1$

$\therefore 3n + 8 = O(n)$ with $c = 4$ and $n_0 = 8$

Example-2 Find upper bound for $f(n) = n^2 + 1$

Solution: $n^2 + 1 \leq 2n^2$, for all $n \geq 1$

$\therefore n^2 + 1 = O(n^2)$ with $c = 2$ and $n_0 = 1$

Example-3 Find upper bound for $f(n) = n^4 + 100n^2 + 50$

Solution: $n^4 + 100n^2 + 50 \leq 2n^4$, for all $n \geq 1$

$\therefore n^4 + 100n^2 + 50 = O(n^4)$ with $c = 2$ and $n_0 = 100$

Example-4 Find upper bound for $f(n) = 2n^3 - 2n^2$

Solution: $2n^3 - 2n^2 \leq 2n^3$, for all $n \geq 1$

$\therefore 2n^3 - 2n^2 = O(2n^3)$ with $c = 2$ and $n_0 = 1$

Example-5 Find upper bound for $f(n) = n$

Solution: $n \leq n^2$, for all $n \geq 1$

$\therefore n = O(n^2)$ with $c = 1$ and $n_0 = 1$

Example-6 Find upper bound for $f(n) = 410$

Solution: $410 \leq 410$, for all $n \geq 1$

$\therefore 100 = O(1)$ with $c = 1$ and $n_0 = 1$

No Uniqueness?

There is no unique set of values for n_0 and c in proving the asymptotic bounds. Let us consider, $100n + 5 = O(n^2)$. For this function there are multiple n_0 and c values possible.

Solution1: $100n + 5 \leq 100n + n = 101n \leq 101n^2$ for all $n \geq 5$, $n_0 = 5$ and $c = 101$ is a solution.

Solution2: $100n + 5 \leq 100n + 5n = 105n \leq 105n^2$ for all $n \geq 1$, $n_0 = 1$ and $c = 105$ is also a solution.

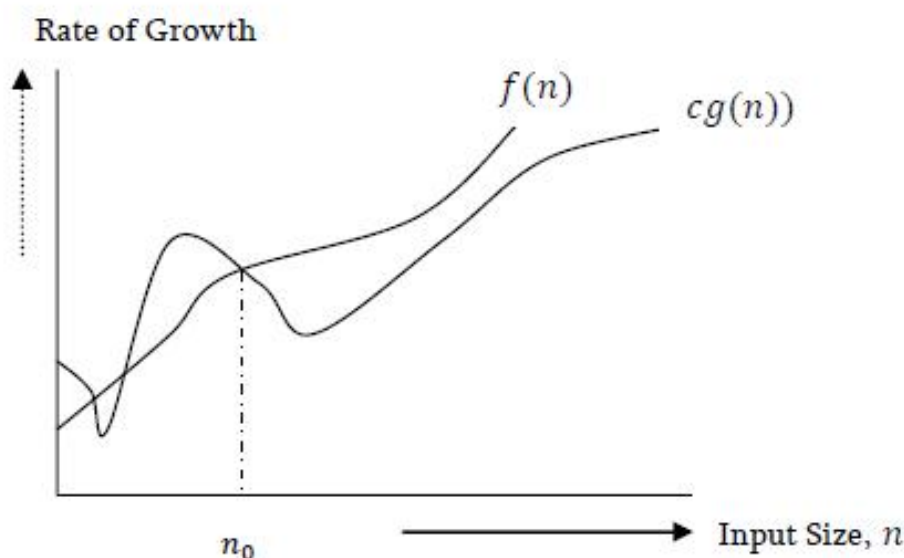
2. Omega Notation

Omega-Ω Notation

Similar to O discussion, this notation gives the tighter lower bound of the given algorithm and we represent it as $f(n) = \Omega(g(n))$. That means, at larger values of n , the tighter lower bound of $f(n)$ is $g(n)$.

For example, if $f(n) = 100n^2 + 10n + 50$, $g(n)$ is $\Omega(n^2)$.

The Ω notation can be defined as $\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$. $g(n)$ is an asymptotic tight lower bound for $f(n)$. Our objective is to give largest rate of growth $g(n)$ which is less than or equal to given algorithms rate of growth $f(n)$.



Ω Examples

Example-1 Find lower bound for $f(n) = 5n^2$

Solution: $\exists c, n_0$ Such that: $0 \leq cn \leq 5n^2 \Rightarrow cn \leq 5n^2 \Rightarrow c = 1$ and $n_0 = 1$
 $\therefore 5n^2 = \Omega(n)$ with $c = 1$ and $n_0 = 1$

Example-2 Prove $f(n) = 100n + 5 \neq \Omega(n^2)$

Solution: $\exists c, n_0$ Such that: $0 \leq cn^2 \leq 100n + 5$
 $100n + 5 \leq 100n + 5n (\forall n \geq 1) = 105n$

$$cn^2 \leq 105n \Rightarrow n(cn - 105) \leq 0$$

$$\text{Since } n \text{ is positive } \Rightarrow cn - 105 \leq 0 \Rightarrow n \leq 105/c$$

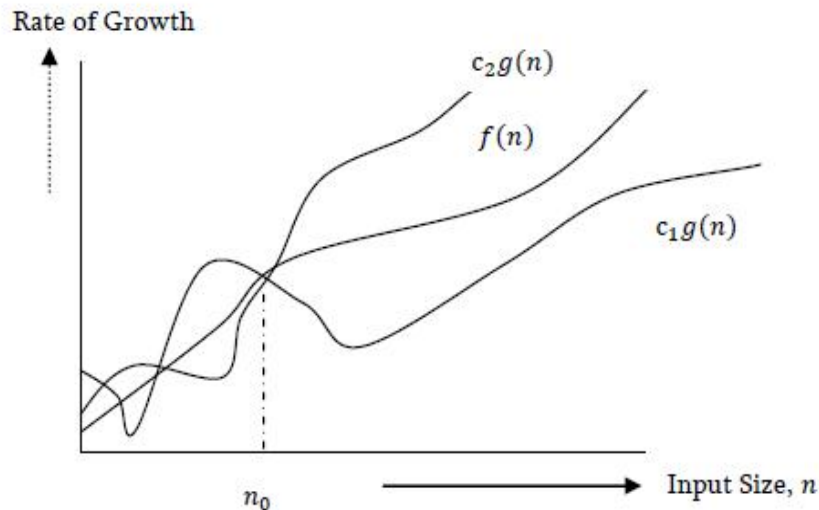
$\Rightarrow$ Contradiction: n cannot be smaller than a constant

Example-3 $n = \Omega(2n)$, $n^3 = \Omega(n^2)$, $n = \Omega(\log n)$

3. Theta- Notation

Theta- θ Notation

This notation decides whether the upper and lower bounds of a given function (algorithm) are same or not. The average running time of algorithm is always between lower bound and upper bound. If the upper bound (O) and lower bound (Ω) gives the same result then θ notation will also have the same rate of growth. As an example, let us assume that $f(n) = 10n + n$ is the expression. Then, its tight upper bound $g(n)$ is $O(n)$. The rate of growth in best case is $g(n) = O(n)$. In this case, rate of growths in best case and worst are same. As a result, the average case will also be same. For a given function (algorithm), if the rate of growths (bounds) for O and Ω are not same then the rate of growth θ case may not be same.



Now consider the definition of θ notation. It is defined as $\theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\}$. $g(n)$ is an asymptotic tight bound for $f(n)$. $\theta(g(n))$ is the set of functions with the same order of growth as $g(n)$.

⊕ Examples

Example-1 Find θ bound for $f(n) = \frac{n^2}{2} - \frac{n}{2}$

Solution: $\frac{n^2}{5} \leq \frac{n^2}{2} - \frac{n}{2} \leq n^2$, for all, $n \geq 1$
 $\therefore \frac{n^2}{2} - \frac{n}{2} = \theta(n^2)$ with $c_1 = 1/5, c_1 = 1$ and $n_0 = 1$

Example-2 Prove $n \neq \theta(n^2)$

Solution: $c_1 n^2 \leq n \leq c_2 n^2 \Rightarrow$ only holds for: $n \leq 1/c_1$
 $\therefore n \neq \Theta(n^2)$

Example-3 Prove $6n^3 \neq \theta(n^2)$

Solution: $c_1 n^2 \leq 6n^3 \leq c_2 n^2 \Rightarrow$ only holds for: $n \leq c_2/6$
 $\therefore 6n^3 \neq \Theta(n^2)$

Example-4 Prove $n \neq \theta(\log n)$

Solution: $c_1 \log n \leq n \leq c_2 \log n \Rightarrow c_2 \geq \frac{n}{\log n}, \forall n \geq n_0$ – Impossible

Important Notes

For analysis (best case, worst case and average) we try to give upper bound (O) and lower bound (Ω) and average running time (θ). From the above examples, it should also be clear that, for a given function (algorithm) getting upper bound (O) and lower bound (Ω) and average running time (θ) may not be possible always. For example, if we are discussing the best case of an algorithm, then we try to give upper bound (O) and lower bound (Ω) and average running time (θ).

In the remaining chapters we generally concentrate on upper bound (O) because knowing lower bound (Ω) of an algorithm is of no practical importance and we use θ notation if upper bound (O) and lower bound (Ω) are same.

Why is it called Asymptotic Analysis?

From the above discussion (for all the three notations: worst case, best case and average case), we can easily understand that, in every case for a given function $f(n)$ we are trying to find other function $g(n)$ which approximates $f(n)$ at higher values of n . That means, $g(n)$ is also a curve which approximates $f(n)$ at higher values of n . In mathematics we call such curve as *asymptotic curve*. In other terms, $g(n)$ is the asymptotic curve for $f(n)$. For this reason, we call algorithm analysis as *asymptotic analysis*.

Questions

Q1. You have two algorithms that solve the same problem:

Algorithm X has time complexity ' $O(n \log n)$ '

Algorithm Y has time complexity ' $O(n^2)$ '

But when implemented, you observe that for small values of ' n ' (say ' $n < 100$ '), Algorithm Y actually runs faster.

Explain why this can happen, and what should guide your choice of algorithm.

Q2. A developer writes a function that takes ' $O(n^2)$ ' time in the worst case, but they found that on their dataset it mostly runs in linear time.

What could be the reason for this? Give an example of such a situation.

Q3. You have an algorithm whose time complexity is expressed as:

$$T(n) = 3n \log n + 20n + 1000 \log n$$

- What is its asymptotic time complexity using Big-O notation?
- Justify each simplification step.

Q4. Given two functions $f(n) = n^2$ and $g(n) = n \log n$, show formally

why: $f(n) \in \Omega(g(n))$

Provide a small mathematical inequality or explanation.

Q5. True or False:

.If $f(n) \in O(g(n))$ then necessarily $g(n) \in \Omega(f(n))$

Q6. Why is Big-Theta (Θ) notation considered stronger than Big-O or Big-Omega?

Q7. The recurrence relation for a recursive algorithm is given by:

$$T(n) = 2T(n/2) + n$$

- a) Solve this recurrence using the Master Theorem.
- b) State the time complexity.

Q8. A friend says:

"My algorithm is $O(n^3)$. This means it will always run slower than your $O(n^2)$ algorithm"

Do you agree? Why or why not?

Q9. Consider two nested loops:

```
for i in range(n):  
    for j in range(2*n):  
        print(i, j)
```

What is the time complexity? Show your reasoning step by step.

Q10. Discuss why average-case analysis can be more difficult or even misleading compared to worst-case analysis. Give an example.

CHAPTER 3: Mathematical Analysis of Nonrecursive Algorithms and Recursive Algorithms

3.1 Mathematical Analysis of Nonrecursive Algorithms

Nonrecursive algorithms are the most basic form of algorithms in computer science. They execute in a **linear or iterative manner** (usually through loops), without calling themselves recursively.

The goal of mathematically analyzing these algorithms is to evaluate their **time efficiency**, which refers to how the execution time of an algorithm grows with respect to the size of its input. This kind of analysis helps us predict algorithm behavior **independently of hardware or software environments**.

Purpose of the Analysis

- To understand **how efficiently an algorithm performs** as the input size increases.
- To estimate the **number of operations** performed for different input cases (best, worst, average).
- To **compare multiple algorithms** and choose the most suitable one.

Why Use Mathematical Analysis?

While we can measure execution time empirically (e.g., using timers), the results are affected by machine speed, compiler optimizations, and operating system.

Instead, mathematical analysis gives an **abstract, machine-independent measure** of algorithm performance that focuses purely on **growth rate** as input size increases.

Basic Summation Rules

1. Summation of a Constant

$$\sum_{i=l}^u 1 = u - l + 1$$

Explanation:

- This formula represents the sum of the constant 1 over a range of integer values from l (lower bound) to u (upper bound).
- Since each term is just 1, the result is simply the number of terms in the range.

Reasoning:

If you are summing 1 repeatedly for each index i , then:

$$1 + 1 + 1 + \cdots + 1 = (\text{number of terms}) = u - l + 1$$

Example:

$$\sum_{i=3}^7 1 = 1 + 1 + 1 + 1 + 1 = 5 = 7 - 3 + 1$$

Usage in Algorithm Analysis:

This rule is used when a loop executes a **constant-time operation** (e.g., assignment, increment) a certain number of times — for example:

```
for (i = 1; i <= n; i++)  
    x++;
```

Here, the basic operation (`x++`) executes n times, so:

$$T(n) = \sum_{i=1}^n 1 = n$$

→ Time Complexity: $O(n)$

2. Summation of i (Linear Series)

$$\sum_{i=l}^u i = \frac{(u+l)(u-l+1)}{2}$$

Explanation:

This formula represents the **sum of consecutive integers** from **l (lower bound)** to **u (upper bound)**.

It gives the total when you add all numbers starting from l up to u .

Derivation (Idea):

The sum of the first n integers (from 1 to n) is well known:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

To generalize for any range l to u :

$$\begin{aligned}\sum_{i=l}^u i &= \sum_{i=1}^u i - \sum_{i=1}^{l-1} i \\ &= \frac{u(u+1)}{2} - \frac{(l-1)l}{2} \\ &= \frac{(u+l)(u-l+1)}{2}\end{aligned}$$

Example:

$$\sum_{i=3}^7 i = 3 + 4 + 5 + 6 + 7 = 25$$

Check with formula:

$$\frac{(7 + 3)(7 - 3 + 1)}{2} = \frac{10 \times 5}{2} = 25$$

✅ Works perfectly!

Usage in Algorithm Analysis:

This summation appears when the **number of operations increases linearly** with the loop index — for example:

```
for (i = 1; i <= n; i++)  
    for (j = 1; j <= i; j++)  
        x++;
```

Here:

- The **inner loop** executes i times for each i .
- So, total operations:

$$T(n) = \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

→ Time Complexity: $O(n^2)$

3. Summation of Squares

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Explanation:

This gives the sum of the **squares of the first n integers**.

Example:

$$\sum_{i=1}^4 i^2 = 1^2 + 2^2 + 3^2 + 4^2 = 30$$

Check using the formula:

$$\frac{4(4+1)(2 \times 4 + 1)}{6} = \frac{4 \times 5 \times 9}{6} = 30$$

✅ Correct!

Usage in Algorithm Analysis:

Used when the **number of operations** depends on i^2 — for example:

```
cpp
for (i = 1; i <= n; i++)
    for (j = 1; j <= i*i; j++)
        x++;
```

Here, the inner loop executes i^2 times, so:

$$T(n) = \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

→ Time Complexity: $O(n^3)$

4. Summation of Cubes

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

Explanation:

This gives the sum of the **cubes of the first n integers**, and interestingly, it equals the **square of the sum of the first n integers**.

Example:

$$\sum_{i=1}^3 i^3 = 1^3 + 2^3 + 3^3 = 36$$

Check:

$$\left(\frac{3(3+1)}{2} \right)^2 = \left(\frac{12}{2} \right)^2 = 6^2 = 36$$

✅ Correct again!

Usage in Algorithm Analysis:

Appears in algorithms where the loop count depends on i^3 , for example:

```
cpp

for (i = 1; i <= n; i++)
    for (j = 1; j <= i*i; j++)
        for (k = 1; k <= i; k++)
            x++;
```

Here:

$$T(n) = \sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

→ Time Complexity: $O(n^4)$

Summary Table

Formula Type	Mathematical Expression	Growth Order
Constant	$\sum 1 = n$	$O(n)$
Linear	$\sum i = \frac{n(n+1)}{2}$	$O(n^2)$
Quadratic	$\sum i^2 = \frac{n(n+1)(2n+1)}{6}$	$O(n^3)$
Cubic	$\sum i^3 = \left(\frac{n(n+1)}{2} \right)^2$	$O(n^4)$

Example 1: Finding the Maximum Element in an Array

EXAMPLE 1 Consider the problem of finding the value of the largest element in a list of n numbers. For simplicity, we assume that the list is implemented as an array. The following is pseudocode of a standard algorithm for solving the problem.

ALGORITHM *MaxElement*($A[0..n-1]$)

//Determines the value of the largest element in a given array

//Input: An array $A[0..n-1]$ of real numbers

//Output: The value of the largest element in A

$maxval \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n-1$ **do**

if $A[i] > maxval$

$maxval \leftarrow A[i]$

return $maxval$

Analysis Steps

- **Input Size:** n (number of elements in array A)
- **Basic Operation:** Comparison $A[i] > \text{maxval}$ (happens every iteration)
- **Case Dependency:** All cases (worst, best, average) have same number of comparisons $\rightarrow$ **no branching/exit**
- **Set Up the Sum:**
 - Loop runs from $i = 1$ to $n - 1$
 - So, the number of comparisons =

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1$$

Time Complexity

Number of comparisons: $C(n)=n-1$

Hence, time complexity:

$$C(n) \in \Theta(n)$$

EXAMPLE 2 Consider the *element uniqueness problem*: check whether all the elements in a given array of n elements are distinct. This problem can be solved by the following straightforward algorithm.

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

Analysis Steps

- **Input Size:** n (number of elements)
- **Basic Operation:** Comparison $A[i] == A[j]$
- **Case Dependency:** Varies depending on array content, so we analyze the **worst case** (all elements distinct $\rightarrow$ no early termination)
- **Set Up the Sum:**

Outer loop: $i = 0$ to $n - 2$ (runs $n - 1$ times)

Inner loop: $j = i + 1$ to $n - 1$ (runs $n - 1 - i$ times for each i)

So, the total number of comparisons is:

$$C_{\text{worst}}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i)$$

Simplify the sum:

$$= (n-1) + (n-2) + \cdots + 1 = \sum_{k=1}^{n-1} k = \frac{(n-1) \cdot n}{2}$$

Time Complexity

$$C_{\text{worst}}(n) = \frac{n(n-1)}{2} \in \Theta(n^2)$$

General Plan for Analyzing the Time Efficiency of Nonrecursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.
2. Identify the algorithm's basic operation. (As a rule, it is located in the innermost loop.)
3. Check whether the number of times the basic operation is executed depends only on the size of an input. If it also depends on some additional property, the worst-case, average-case, and, if necessary, best-case efficiencies have to be investigated separately.
4. Set up a sum expressing the number of times the algorithm's basic operation is executed.
5. Using standard formulas and rules of sum manipulation, either find a closedform formula for the count or, at the very least, establish its order of growth.

3.2 Mathematical Analysis of Recursive Algorithms

Recursive algorithms are algorithms that solve a problem by **calling themselves** on smaller instances of the same problem. They are widely used in computer science for tasks such as **divide and conquer**, **backtracking**, **dynamic programming**, and **tree/graph traversal**.

Mathematically analyzing recursive algorithms is essential to understand how **efficient** they are in terms of **time and space** as the input size increases.

Purpose of the Analysis

- To determine how many **recursive calls** are made.
- To understand how the **input size is reduced** at each step.
- To establish the **overall time complexity** by solving a **recurrence relation** that models the algorithm's behavior.
- To compare recursive solutions with iterative alternatives.

Why Analyze Recursive Algorithms Separately?

Unlike nonrecursive algorithms, recursive ones often involve:

- **Multiple recursive calls**
- **Varying input sizes** across different branches
- **Overlapping subproblems**

These characteristics make recursive algorithms harder to analyze using just simple loops. Instead, we usually model their time complexity using **recurrence relations**, then solve or approximate them using mathematical techniques.

In this section, we learn how to analyze **recursive algorithms** using the **general framework** for algorithm analysis. This process involves:

- Identifying the input size

- Determining the basic operation
- Setting up a **recurrence relation**
- Solving the recurrence to find time complexity

EXAMPLE 1 Compute the factorial function $F(n) = n!$ for an arbitrary nonnegative integer n . Since

$$n! = 1 \cdot \dots \cdot (n-1) \cdot n = (n-1)! \cdot n \quad \text{for } n \geq 1$$

and $0! = 1$ by definition, we can compute $F(n) = F(n-1) \cdot n$ with the following recursive algorithm.

ALGORITHM $F(n)$

```
//Computes  $n!$  recursively
//Input: A nonnegative integer  $n$ 
//Output: The value of  $n!$ 
if  $n = 0$  return 1
else return  $F(n-1) * n$ 
```

We can define it **recursively** as:

- ✓ **Base case:** $F(0) = 1$
- ✓ **Recursive case:** $F(n) = F(n-1) \cdot n$ for $n \geq 1$

Analyzing the Time Efficiency

1. Input size:

We consider n (not the number of bits in n , just the integer value itself).

2. Basic Operation:

The multiplication $F(n-1) \cdot n$ is the main operation we want to count.

$M(n)$ be the number of multiplications performed when computing $F(n)$.

Setting Up the Recurrence Relation

Each recursive call does one multiplication:

- ✓ To compute $F(n)$ we first compute $F(n-1) \rightarrow$ this requires $M(n-1)$ multiplications
- ✓ Then we multiply the result by $n \rightarrow$ 1 more multiplication

So:

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0$$

But we need an initial condition:

When $n=0$, the algorithm **returns 1 without any multiplication**

So, $M(0)=0$

The Recurrence System

$$\begin{cases} M(n) = M(n - 1) + 1, & \text{for } n > 0 \\ M(0) = 0 \end{cases}$$

Solving the Recurrence

Let's solve it by unrolling:

$$\begin{aligned} M(n) &= M(n - 1) + 1 \\ &= M(n - 2) + 1 + 1 \\ &= M(n - 3) + 1 + 1 + 1 \\ &\dots \\ &= M(0) + n \\ &= 0 + n = n \end{aligned}$$

Time Complexity

- Number of multiplications: $M(n) = n$
- Therefore, the time complexity is:

$$\boxed{\Theta(n)}$$

General Plan for Analyzing the Time Efficiency of Recursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.
2. Identify the algorithm's basic operation.
3. Check whether the number of times the basic operation is executed can vary on different inputs of the same size; if it can, the worst-case, average-case, and best-case efficiencies must be investigated separately.
4. Set up a recurrence relation, with an appropriate initial condition, for the number of times the basic operation is executed.
5. Solve the recurrence or, at least, ascertain the order of growth of its solution.

EXAMPLE 3 As our next example, we investigate a recursive version of the algorithm discussed at the end of Section 2.3.

ALGORITHM *BinRec*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

if $n = 1$ **return** 1

else return *BinRec*($\lfloor n/2 \rfloor$) + 1

Algorithm: BinRec(n)

Input: A positive integer (n)

Output: The number of binary digits (bits) in the binary representation of (n)

Idea of the algorithm

Every time we divide (n) by 2 (taking the floor), we remove one binary digit.

When we reach ($n = 1$), only one binary digit remains.

So, the number of binary digits in (n) equals the number of times we can divide (n) by 2 until we reach 1, plus one.

Recurrence for the number of additions

Let ($A(n)$) denote the **number of additions** performed by the algorithm.

* To compute $\text{BinRec}(\lfloor n/2 \rfloor)$, the number of additions is ($A(\lfloor n/2 \rfloor)$).

* The algorithm then performs one extra addition (the '+1' in the return statement).

Hence, the recurrence is:

$$A(n) = A(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1$$

and the base case:

$$A(1) = 0$$

(since no additions occur when ($n = 1$)).

Solving the recurrence (for powers of two)

To make analysis easier, we assume ($n = 2^k$).

Then the recurrence becomes:

$$A(2^k) = A(2^{k-1}) + 1$$

with the initial condition:

$$A(2^0) = 0$$

Now apply **backward substitution:**

$$\begin{aligned} A(2^k) &= A(2^{k-1}) + 1 \\ &= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2 \\ &= A(2^{k-3}) + 3 \\ &\vdots \\ &= A(2^0) + k = 0 + k = k \end{aligned}$$

Final result

Since $n = 2^k$, we have $k = \log_2 n$.

Therefore:

$$A(n) = \log_2 n$$

This means the number of additions — and hence the time complexity — grows logarithmically with (n).

Conclusion

Number of recursive calls = number of binary digits in $n = \lfloor \log_2 n \rfloor + 1$

Number of additions $A(n) = \lfloor \log_2 n \rfloor$

Time complexity: ($T(n) \in \Theta(\log n)$)

Step-by-Step Explanation of BinRec(n)

Step 1: Write the algorithm

```
Algorithm BinRec(n)
// Input: A positive integer n
// Output: Number of binary digits in n
if n = 1
    return 1
else
    return BinRec(L n/2 J ) + 1
```

Step 2: Understand what it does

Each time the algorithm divides (n) by 2 (using integer division), it removes one binary digit.

The recursion continues until (n = 1).

So, the **number of recursive calls** (or the number of divisions by 2) equals the number of bits in (n).

Step 3: Identify the recurrence relation

We define (A(n)) = number of **additions** performed by the algorithm.

The recursive call BinRec(L n/2 J) performs (A(L n/2 J)) additions.

After the recursive call, the algorithm adds 1 (+1).

Hence:

$$A(n) = A(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1$$

Step 4: Simplify for powers of 2

To make the recurrence easier to solve, assume (n = 2^k).

Then:

$$A(2^k) = A(2^{k-1}) + 1$$

$$A(2^0) = 0$$

Step 5: Use backward substitution

Now we substitute repeatedly:

$$\begin{aligned} A(2^k) &= A(2^{k-1}) + 1 \\ &= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2 \\ &= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3 \\ &\vdots \\ &= A(2^0) + k = 0 + k \end{aligned}$$

Step 6: Replace (k) with ($\log_2 n$)

Since $n = 2^k \Rightarrow k = \log_2 n$,
we get:

$$A(n) = \log_2 n$$

Step 7: Interpret the result

- ✓ Number of additions: $A(n) = \lfloor \log_2 n \rfloor$
- ✓ Number of recursive calls: $\lfloor \log_2 n \rfloor + 1$
- ✓ Time complexity: $T(n) \in \Theta(\log n)$

Step 8: Example

Let's take ($n = 13$):

Recursive call n

Result

Recursive call n	Result
1	$13 \rightarrow \text{BinRec}(6) + 1$
2	$6 \rightarrow \text{BinRec}(3) + 1$
3	$3 \rightarrow \text{BinRec}(1) + 1$
4	$1 \rightarrow \text{returns } 1$

Now count:

- Number of calls = 4
- Number of additions = 3
- $(\lfloor \log_2 13 \rfloor = 3) \checkmark$ (matches result)

Questions

ALGORITHM *Mystery(n)*

```
//Input: A nonnegative integer  $n$ 
 $S \leftarrow 0$ 
for  $i \leftarrow 1$  to  $n$  do
     $S \leftarrow S + i * i$ 
return  $S$ 
```

- What does this algorithm compute?
- What is its basic operation?
- How many times is the basic operation executed?
- What is the efficiency class of this algorithm?
- Suggest an improvement, or a better algorithm altogether, and indicate its efficiency class. If you cannot do it, try to prove that, in fact, it cannot be done.

5. Consider the following algorithm.

ALGORITHM *Secret*($A[0..n - 1]$)

//Input: An array $A[0..n - 1]$ of n real numbers

$minval \leftarrow A[0]; maxval \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $A[i] < minval$

$minval \leftarrow A[i]$

if $A[i] > maxval$

$maxval \leftarrow A[i]$

return $maxval - minval$

- a. What does this algorithm compute?
- b. What is its basic operation?
- c. How many times is the basic operation executed?
- d. What is the efficiency class of this algorithm?
- e. Suggest an improvement, or a better algorithm altogether, and indicate its efficiency class. If you cannot do it, try to prove that, in fact, it cannot be done.

6. Consider the following recursive algorithm.

ALGORITHM *Riddle*($A[0..n - 1]$)

//Input: An array $A[0..n - 1]$ of real numbers

if $n = 1$ **return** $A[0]$

else $temp \leftarrow Riddle(A[0..n - 2])$

if $temp \leq A[n - 1]$ **return** $temp$

else return $A[n - 1]$

- a. What does this algorithm compute?
- b. Set up a recurrence relation for the algorithm's basic operation count and solve it.

ALGORITHM $S(n)$

//Input: A positive integer n

//Output: The sum of the first n cubes

if $n = 1$ **return** 1

else return $S(n - 1) + n * n * n$

- a. Set up and solve a recurrence relation for the number of times the algorithm's basic operation is executed.
- b. How does this algorithm compare with the straightforward nonrecursive algorithm for computing this sum?

CHAPTER 4: Sorting Algorithms

Sorting is a fundamental operation in computer science, widely used in applications ranging from data organization to efficient searching. In this chapter, all sorting algorithms are demonstrated using specific data structures—commonly 32-bit integers—because their comparison operations (like $<$, $>$, etc.) have well-defined and predictable behavior.

These sorting algorithms, while demonstrated on integers, can be easily adapted to handle generic data types using features available in modern programming languages. The focus here is on understanding the logic, efficiency, and behavior of the sorting techniques, which can be applied across various contexts regardless of the data type involved.

4.1-Bubble Sort

Bubble Sort is a simple comparison-based sorting algorithm. It works by **repeatedly swapping adjacent elements** if they are in the wrong order. The process continues until no more swaps are needed — meaning the list is sorted.

How Does It Work?

Imagine a row of numbers. Bubble Sort goes through the list and compares **two numbers at a time**:

- If the **left number is greater** than the right one, they are **swapped**.
- This process is repeated for the **entire list**, multiple times.
- With each full pass, the **largest unsorted element “bubbles up”** to its correct position at the end.

Step-by-Step Example

Let's sort this list in **ascending order**:

Initial list: [5, 1, 4, 2, 8]

Pass 1:

Compare 5 and 1 $\rightarrow$ swap $\rightarrow$ [1, 5, 4, 2, 8]

Compare 5 and 4 $\rightarrow$ swap $\rightarrow$ [1, 4, 5, 2, 8]

Compare 5 and 2 $\rightarrow$ swap $\rightarrow$ [1, 4, 2, 5, 8]

Compare 5 and 8 $\rightarrow$ no swap

✓ Largest element (8) is now at the end.

Pass 2:

Compare 1 and 4 $\rightarrow$ no swap

Compare 4 and 2 $\rightarrow$ swap $\rightarrow$ [1, 2, 4, 5, 8]

Compare 4 and 5 $\rightarrow$ no swap

Compare 5 and 8 $\rightarrow$ no swap

✓ Second-largest (5) is in place.

Pass 3:

Compare 1 and 2 $\rightarrow$ no swap

Compare 2 and 4 $\rightarrow$ no swap

Compare 4 and 5 $\rightarrow$ no swap

No swaps? ✓ List is fully sorted.

Final list: [1, 2, 4, 5, 8]

Table 1. Time and Space Complexity

Case	Time Complexity	Why?
Best case	$O(n)$	Already sorted; only one pass needed
Average case	$O(n^2)$	Needs multiple passes
Worst case	$O(n^2)$	Reversed list, max comparisons/swaps
Space	$O(1)$	In-place sort; no extra memory used

✓ Advantages

Very easy to understand and implement.

Good for teaching sorting concepts.

✗ Disadvantages

Very slow for large datasets.

Not used in real-world applications due to inefficiency.

Pseudocode

```
1) algorithm BubbleSort(list)
2)   Pre: list  $\neq \emptyset$ 
3)   Post: list has been sorted into values of ascending order
4)   for i  $\leftarrow$  0 to list.Count - 1
5)     for j  $\leftarrow$  0 to list.Count - 1
6)       if list[i] < list[j]
7)         Swap(list[i], list[j])
8)       end if
9)     end for
10)  end for
11)  return list
12) end BubbleSort
```

Bubble Sort Pseudocode Analysis

1. This line defines the algorithm named BubbleSort, which takes an input parameter called list. This is the array or list of elements to be sorted.
2. The **precondition** specifies that the list must **not be empty**. There should be at least one element to sort.
3. The **postcondition** ensures that by the end of the algorithm, the list will be **sorted in ascending order** (from smallest to largest).
4. This is the **outer loop**.
It runs from $i = 0$ to $i = n - 1$, where n is the number of elements in the list.
This loop controls the **number of passes** over the entire list.
5. This is the **inner loop**, nested inside the outer one.
It also runs from $j = 0$ to $j = n - 1$.
This loop is used to **compare pairs of elements** and swap them if they are out of order.
6. This condition checks whether the element at position i is **less than** the element at j .

Note: This condition is **not correct** for standard Bubble Sort. Usually, Bubble Sort compares **adjacent elements**, like if $\text{list}[j] > \text{list}[j+1]$.

So this version of the code is **flawed**.

7. If the condition in line 6 is true, the algorithm swaps $\text{list}[i]$ and $\text{list}[j]$.

Again, this logic does not match the typical Bubble Sort, which only swaps **adjacent** elements.

8. end if

9. end for

10. end for

These lines just close the inner and outer loops.

11. return list

12. end BubbleSort

13. The sorted list is returned after all comparisons and swaps are done.

! Correction to the Bubble Sort Logic

A more correct version of Bubble Sort is:

for $i \leftarrow 0$ to $n - 1$

 for $j \leftarrow 0$ to $n - i - 2$

 if $\text{list}[j] > \text{list}[j + 1]$

 swap($\text{list}[j]$, $\text{list}[j + 1]$)

This way, larger elements “bubble” to the end of the list with each pass.

Time Complexity Analysis

Let’s analyze how many **comparisons** and **swaps** are done:

Worst-case Time Complexity:

Two nested loops:

Outer loop: runs n times.

Inner loop: in worst case, runs n - 1 times for each i.

So total comparisons:

$$(n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2}$$

This leads to:

- $O(n^2)$ time complexity in worst and average cases

time complexity in worst and average cases

Best-case Time Complexity:

If the list is already sorted, and we optimize by checking for no swaps, the best case becomes: $O(n)$ (with optimization)

4.2 Merge Sort

What is Merge Sort?

Merge Sort is a **Divide and Conquer** sorting algorithm.

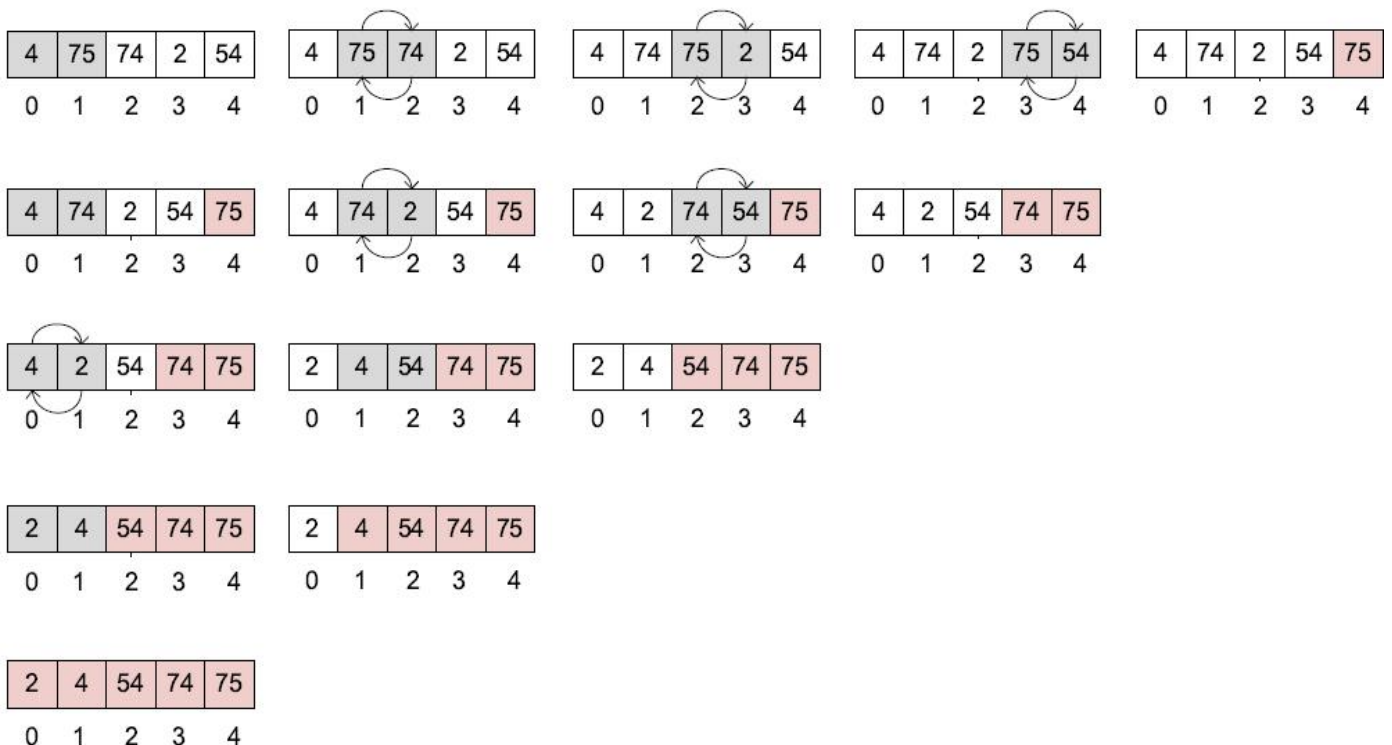
It works by:

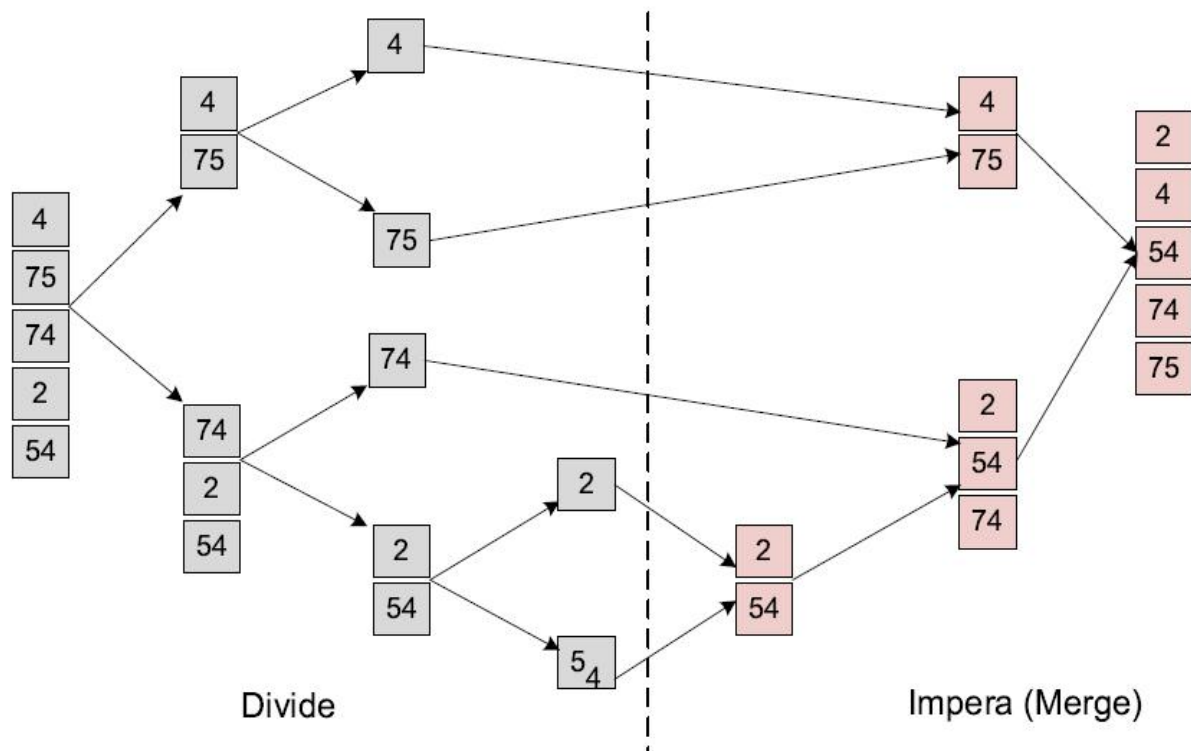
- **Dividing** the list into two halves,
- **Recursively sorting** each half,
- And **merging** the sorted halves into a single sorted list.

```

1) algorithm Mergesort(list)
2)   Pre: list  $\neq \emptyset$ 
3)   Post: list has been sorted into values of ascending order
4)   if list.Count = 1 // already sorted
5)     return list
6)   end if
7)    $m \leftarrow \text{list.Count} / 2$ 
8)   left  $\leftarrow \text{list}(m)$ 
9)   right  $\leftarrow \text{list}(\text{list.Count} - m)$ 
10)  for  $i \leftarrow 0$  to left.Count - 1
11)    left[i]  $\leftarrow \text{list}[i]$ 
12)  end for
13)  for  $i \leftarrow 0$  to right.Count - 1
14)    right[i]  $\leftarrow \text{list}[i]$ 
15)  end for
16)  left  $\leftarrow \text{Mergesort}(\text{left})$ 
17)  right  $\leftarrow \text{Mergesort}(\text{right})$ 
18)  return MergeOrdered(left, right)
19) end Mergesort

```





How Merge Sort Works – Step-by-Step

Suppose we want to sort this list:

[6, 3, 8, 5, 2, 7, 4, 1]

1- Divide

We split the list into two halves:

Left: [6, 3, 8, 5]

Right: [2, 7, 4, 1]

Then we split again until each sublist has **one element**:

[6, 3, 8, 5] → [6, 3] → [6] [3] → [8, 5] → [8] [5]

[2, 7, 4, 1] → [2, 7] → [2] [7] → [4, 1] → [4] [1]

2- Conquer (Sort and Merge)

Now we **merge** each pair in **sorted order**:

[6] [3] $\rightarrow$ [3, 6]

[8] [5] $\rightarrow$ [5, 8] $\rightarrow$ [3, 6] [5, 8] $\rightarrow$ [3, 5, 6, 8]

[2] [7] $\rightarrow$ [2, 7]

[4] [1] $\rightarrow$ [1, 4] $\rightarrow$ [2, 7] [1, 4] $\rightarrow$ [1, 2, 4, 7]

3- Merge Final Sorted Halves

[3, 5, 6, 8] and [1, 2, 4, 7] $\rightarrow$ [1, 2, 3, 4, 5, 6, 7, 8]

The list is now fully sorted!

Merge Sort Pseudocode Explanation

- 1) This defines a recursive algorithm named Mergesort that takes a list as input.
- 2) The **precondition** is that the list is not empty — you cannot sort an empty list meaningfully.
- 3) The **postcondition** is that the function will return a sorted list in **ascending** order.
- 4) if list.Count = 1 // already sorted
- 5) return list
- 6) end if

If the list has **only one element**, it is already sorted.

So we just **return** it without any further processing.

This is the **base case** of the recursion.

- 7) Calculate the **middle index** m of the list to divide it into two halves.

8) left $\leftarrow$ list(m)

9) right $\leftarrow$ list(list.Count - m)

left and right are placeholders to hold the two **sub-lists**.

left will have the first m elements, right will have the remaining $n - m$.

However, these lines are **syntactically unclear**. The actual copying happens in lines 10–15.

10) for $i \leftarrow 0$ to left.Count - 1

11) left[i] $\leftarrow$ list[i]

12) end for

Copy the first half of list into the left list.

13) for $i \leftarrow 0$ to right.Count - 1

14) right[i] $\leftarrow$ list[i]

15) end for

Copy the second half of list into the right list.

Note: The index i here starts from 0, but should access list[$m + i$]. Otherwise, it's a logic error.

16) left $\leftarrow$ Mergesort(left)

17) right $\leftarrow$ Mergesort(right)

Recursively sort the left and right sublists.

18) return MergeOrdered(left, right)

Merge the two sorted halves using a helper function MergeOrdered, which combines them into one **fully sorted** list.

What is MergeOrdered?

This helper function:

- Takes **two sorted lists**.

- Compares the elements **one by one**.
- Merges them into a new list in **sorted order**.

It works like the **merge step** in the Merge Sort algorithm.

Time Complexity Analysis

Let's break down the time complexity:

Recursive Depth:

Each time, the list is divided into **two halves**, so the **depth** of recursion is: $\log_2 n$

Work Per Level:

At each recursive level, merging two halves takes **$O(n)$** time in total.

Total Time: $T(n)=O(n \log n)$

Time Complexity

Merge Sort always has a time complexity of:

Best case: $O(n \log n)$

Average case: $O(n \log n)$

Worst case: $O(n \log n)$

Why?

Splitting the list takes **$\log n$** levels (like a binary tree).

At each level, **merging takes $O(n)$** total time.

So: Total Time= $O(n \log n)$

Space Complexity

Merge Sort uses **extra space** to store the divided sublists:

Space Complexity = $O(n)$

✓ Advantages of Merge Sort

- ✓ Very **efficient** for large datasets.
- ✓ Has **consistent performance** regardless of input.
- ✓ **Stable sort** (preserves the order of equal elements).
- ✓ Used in external sorting (e.g., sorting data on disk).

Disadvantages

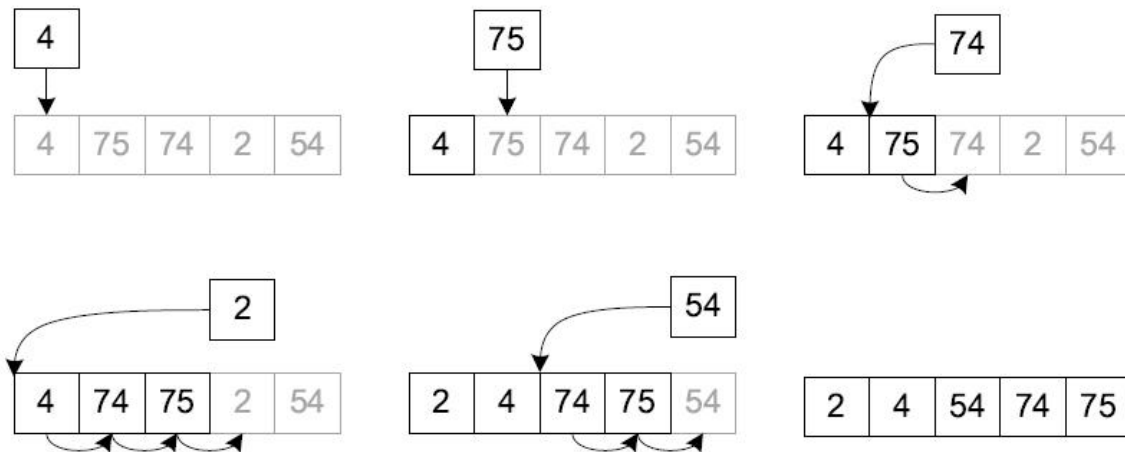
- ✓ Uses **extra memory** (not in-place).
- ✓ Slower for small lists compared to simple algorithms like Insertion Sort.

4.3 Insertion Sort Algorithm

Insertion Sort is a simple sorting algorithm that builds the final sorted list one element at a time. It works similarly to how you sort playing cards in your hand — you pick one card and insert it into the correct position in the sorted portion.

Insertion Sort mimics the way you sort playing cards in your hand:

- Start with one card.
- Pick the next card, and insert it into its correct position among the sorted ones.
- Repeat until the entire hand is sorted.



```

1) algorithm Insertionsort(list)
2)   Pre:  $list \neq \emptyset$ 
3)   Post: list has been sorted into values of ascending order
4)   unsorted  $\leftarrow 1$ 
5)   while unsorted < list.Count
6)     hold  $\leftarrow list[unsorted]$ 
7)     i  $\leftarrow unsorted - 1$ 
8)     while  $i \geq 0$  and  $hold < list[i]$ 
9)        $list[i + 1] \leftarrow list[i]$ 
10)       $i \leftarrow i - 1$ 
11)    end while
12)     $list[i + 1] \leftarrow hold$ 
13)    unsorted  $\leftarrow unsorted + 1$ 
14)  end while
15)  return list
16) end Insertionsort

```

Algorithm: Insertionsort(list)

Line 1) algorithm Insertionsort(list)

This is the function declaration.

Line 2) Pre: $list \neq \emptyset$

The list is not empty (has elements to sort).

Line 3) Post: list has been sorted in ascending order

After sorting, all elements will be arranged from smallest to largest.

Line 4) $\text{unsorted} \leftarrow 1$

We assume the **first element (index 0)** is already "sorted".

We start sorting from the second element (index 1).

Line 5) while $\text{unsorted} < \text{list.Count}$

Loop through the unsorted elements of the list, starting at index 1.

Line 6) $\text{hold} \leftarrow \text{list}[\text{unsorted}]$

Store the current value (element to be inserted into the correct position) in a temporary variable hold.

Line 7) $i \leftarrow \text{unsorted} - 1$

We start comparing from the last sorted element.

Line 8) while $i \geq 0$ and $\text{hold} < \text{list}[i]$

As long as we haven't reached the beginning of the list and hold is less than the current item:

It means we need to shift the current item right to make space for hold.

Line 9) $\text{list}[i + 1] \leftarrow \text{list}[i]$

Move the current element one position to the right.

Line 10) $i \leftarrow i - 1$

Move one position to the left in the list.

Line 11) end while

End of the inner shifting loop.

Line 12) $\text{list}[i + 1] \leftarrow \text{hold}$

Insert the hold element into its correct position.

Line 13) `unsorted ← unsorted + 1`

Move to the next unsorted element.

Line 14) `end while`

End of the outer loop (continues until all elements are sorted).

Line 15) `return list`

The list is now sorted in ascending order.

Table 2. Time Complexity Analysis:

Case	Explanation	Time
Best	Already sorted list – only one comparison per element	$O(n)$
Average	Half elements need shifting on average	$O(n^2)$
Worst	Reverse order – every element must be compared and shifted	$O(n^2)$

For each element i , we may compare with up to $i-1$ previous elements

$$T(n) = 1 + 2 + 3 + \dots + (n - 1) = \frac{n(n - 1)}{2} = O(n^2)$$

So:

Best Case: Already Sorted List

Inner loop (while) runs **only once** for each outer loop iteration.

Comparisons: **$n - 1$**

Shifts: **0**

➤ Time Complexity: **$O(n)$**

Average Case: Random Order

On average, each element compares with **half** of the sorted portion.

Comparisons: approx.

$$\frac{1}{2} \cdot \frac{n(n-1)}{2} = \frac{n^2}{4}$$

➤ Time Complexity: **$O(n^2)$**

Worst Case: Reverse Sorted List

- Every new element is smaller than all sorted ones.
- So it compares and shifts through the entire sorted portion.

Let's compute:

- First element: 0 comparisons
- Second element: 1 comparison
- Third: 2 comparisons
- ...
- Last (n-th): n-1 comparisons

➤ Total comparisons (and shifts):

$$1 + 2 + 3 + \dots + (n-1) = \frac{n(n-1)}{2} = O(n^2)$$

$$T(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \Rightarrow O(n^2)$$

Space Complexity:

O(1) extra space (in-place sorting, only uses a temporary variable hold).

Example:

Given: [5, 3, 4, 1]

Step-by-step:

Start with 5 (sorted part)

Compare 3 with 5 → insert 3 before 5 → [3, 5, 4, 1]

Compare 4 → insert between 3 and 5 → [3, 4, 5, 1]

Compare 1 → insert before all → [1, 3, 4, 5]

4.4 Quick Sort Algorithm

Quick Sort is one of the most widely used sorting algorithms and is based on the **divide and conquer** strategy, achieving an average-case time complexity of **O(n log n)**. The algorithm works by selecting a **pivot** element and rearranging the array so that all elements less than the pivot come before it, and all elements greater come after it. This key operation is known as **partitioning**. Quick Sort then recursively applies the same process to the sublists on either side of the pivot. The recursion stops when the sublists have a size of one or zero, at which point they are considered sorted.

Choosing a good pivot—such as the **median element**—is critical to the algorithm's performance. A poor choice can degrade Quick Sort's efficiency to **O(n²)** in the worst case.

Example : Let's apply **Quick Sort** step-by-step to the list: [4, 75, 74, 2, 54]

We'll use the **last element as the pivot** in each step (a common strategy).

Step 1: Pivot = 54

We rearrange the list so that:

- ◆ Numbers less than 54 go to the left.
- ◆ Numbers greater than 54 go to the right.

Before partition: [4, 75, 74, 2, 54]

Partitioning:

Compare 4 → less than 54 → stay left

Compare 75 → greater → move right

Compare 74 → greater → move right

Compare 2 → less than 54 → stay left

After partition: [4, 2] [54] [75, 74]

Pivot 54 is now in its **correct sorted position**.

Step 2: Recursively sort the left part [4, 2]

Pivot = 2

Compare 4 → greater than 2 → move right

Partition: [] [2] [4]

Combined: [2, 4]

Step 3: Recursively sort the right part [75, 74]

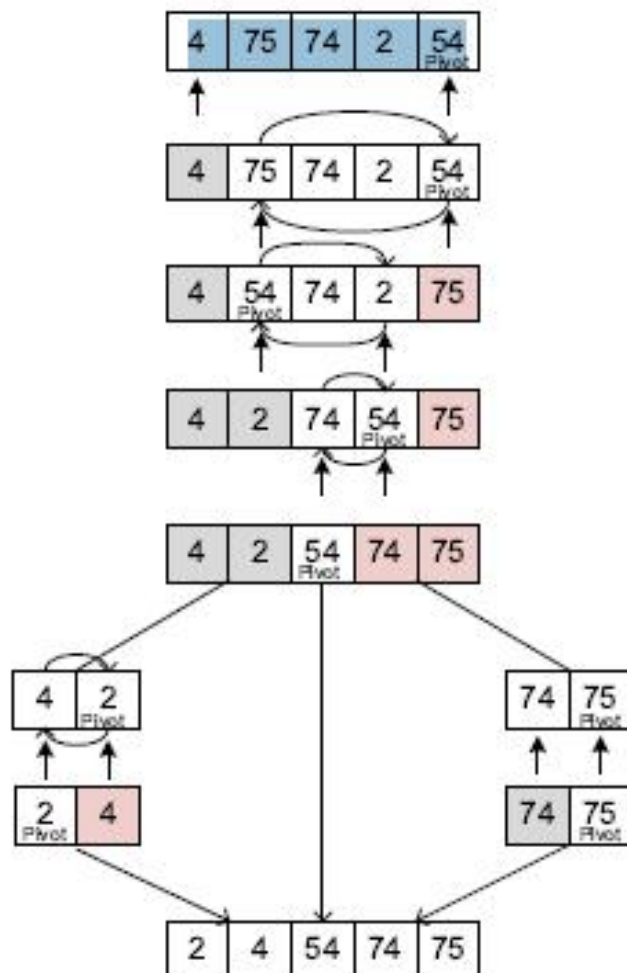
Pivot = 74

Compare 75 → greater than 74 → move right

Partition: [] [74] [75]

Combined: [74, 75]

Final sorted array: Combine all parts: [2, 4] + [54] + [74, 75] = [2, 4, 54, 74, 75]



```

1) algorithm QuickSort(list)
2)   Pre: list  $\neq \emptyset$ 
3)   Post: list has been sorted into values of ascending order
4)   if list.Count = 1 // already sorted
5)     return list
6)   end if
7)   pivot  $\leftarrow$  MedianValue(list)
8)   for i  $\leftarrow$  0 to list.Count - 1
9)     if list[i] = pivot
10)      equal.Insert(list[i])
11)    end if
12)    if list[i] < pivot
13)      less.Insert(list[i])
14)    end if
15)    if list[i] > pivot
16)      greater.Insert(list[i])
17)    end if
18)  end for
19)  return Concatenate(QuickSort(less), equal, QuickSort(greater))
20) end Quicksort

```

Explanation:

Line	What it Does
1	Starts the QuickSort algorithm.
2–3	Precondition: the list is not empty; Postcondition: the list is sorted.
4–6	If the list has 0 or 1 element, return it as it's already sorted.
7	Choose a pivot (ideally the median to avoid worst-case performance).
8–10	Create empty lists for elements less than, equal to, and greater than the pivot.
11–19	Traverse the list and classify each element based on comparison with pivot.
20	Recursively sort the less and greater lists and concatenate the results with the equal list in the middle.
21	End of the algorithm.

Time Complexity of Quick Sort

Case	Time	Explanation
Best Case	$O(n \log n)$	Pivot divides the list perfectly in half each time
Average Case	$O(n \log n)$	Pivot usually gives fairly balanced partitions
Worst Case	$O(n^2)$	Pivot creates highly unbalanced partitions

1. Best Case: $O(n \log n)$ When?

Happens when the pivot always **divides the list into two equal halves** (or close to equal).

For example, if you sort: [3, 6, 4, 5, 2, 7, 1] and pivot = median.

Why $O(n \log n)$?

At each level of recursion:

You process **n elements** (to partition them).

And the depth of the recursion tree is **$\log n$** (since the list is halved each time).

So total work = $n + n/2 + n/4 + \dots = O(n \log n)$

2. Average Case: $O(n \log n)$ When?

Happens in **typical situations** when the pivot is chosen reasonably well — not always perfect, but not worst either.

Real-world inputs usually fall in this case.

Why $O(n \log n)$?

Even if the pivot divides the list **not exactly in half** (say 30%-70%), the recursive calls still don't go very deep.

The total number of comparisons across all levels still ends up around **$n \log n$** .

This is why Quick Sort is fast **in practice**, even though its worst-case is $O(n^2)$.

3. Worst Case: $O(n^2)$ When?

Happens when the pivot is **the smallest or largest element** every time.

This often occurs if:

The input list is already **sorted or reverse sorted**.

The pivot is always chosen as the **first or last element**.

Why $O(n^2)$?

In this case, one partition is of size **$n-1$** and the other is **0** at each recursive call.

So you end up doing:

First level: n comparisons

Second level: $n-1$ comparisons

Third level: $n-2$ comparisons

...

Last level: 1 comparison

So total work = $n + (n-1) + (n-2) + \dots + 1 = \mathbf{O(n^2)}$

Summary in Simple Terms:

Scenario	Shape of Recursion Tree	Depth	Work per Level	Total Work
Best case	Balanced (binary tree)	$\log n$	n	$O(n \log n)$
Average case	Fairly balanced	$\sim \log n$	n	$O(n \log n)$
Worst case	Skewed (linked list shape)	n	decreasing	$O(n^2)$

Tip to Avoid Worst Case:

Use better pivot selection strategies:

Random pivot

Median-of-three (median of first, middle, last elements)

IntroSort: a hybrid that switches to another sort (e.g. HeapSort) when recursion depth gets too large

Questions

Question 1: Analyzing Worst Case of Quick Sort

Explain in detail why Quick Sort may degrade to $O(n^2)$ time complexity in the worst-case scenario. Give a specific example of a list that causes this behavior and describe the steps of partitioning.

Question 2: Quick Sort on Repeated Elements

You are given the list: [4, 4, 4, 4, 4, 4, 4]

Describe what happens when you apply Quick Sort using the **first element as pivot**.

1. How many recursive calls are made?
2. What is the time complexity in this case?

3. Would choosing a random pivot help? Why?

Question 3: Compare Insertion Sort and Quick Sort

For a nearly sorted array of 1000 elements (only 5 elements are out of place), which algorithm performs better — Insertion Sort or Quick Sort? Justify your answer with time complexity and behavior characteristics of both.

Question 4: Quick Sort Stability

Quick Sort is not a stable algorithm by default.

- ✓ What does stability in sorting mean?
- ✓ Why is Quick Sort not stable?
- ✓ Can it be made stable? If yes, how?

Question 5: Median-of-Three Improvement

- ✓ Explain the "**median-of-three**" **pivot strategy** in Quick Sort.
- ✓ Why is it better than choosing the first or last element as pivot?
- ✓ How does it affect the number of comparisons and recursive depth?

CHAPTER 5: SEARCHING ALGORITHMS

Introduction to Searching Algorithms

Searching algorithms are fundamental techniques in computer science used to **locate a specific element** within a data structure such as an array, list, or database. These algorithms play a crucial role in various applications, from retrieving user data in a system to looking up values in a dictionary.

The primary goal of a searching algorithm is to determine **whether a target value exists** in a given dataset and, if so, to **return its position (index)**. If the value does not exist, the algorithm typically returns a special indicator (such as -1 or null).

Why Searching is Important:

Core function in databases, file systems, and software applications.

Enhances data retrieval performance.

Essential for decision-making in algorithms (e.g., pathfinding, AI, search engines).

5.1 Types of Searching Algorithms:

1- Sequential Search Algorithm

Definition:

Sequential Search (also called Linear Search) is a basic searching algorithm that checks each element in a list, one by one, to find a **target value**. It is simple but not very efficient for large datasets.

Purpose: To find the index of a given item in a list. If the item exists, return its index; otherwise, return -1.

How It Works:

- Start from the **first element** of the list.
- Compare the current element with the target item.
- If it matches, **return the index**.
- If not, move to the **next element**.
- Continue until:

A match is found, or

You reach the **end of the list**.

- If no match is found, return **-1**.

Time Complexity:

- ✓ **Best case:** $O(1)$ – the item is the first element.
- ✓ **Worst case:** $O(n)$ – the item is the last element or not in the list.
- ✓ **Average case:** $O(n/2) \approx O(n)$

Algorithm Pseudocode Breakdown:

- 1) algorithm SequentialSearch(list, item)
- 2) Pre: list $\neq \emptyset$ // Make sure the list is not empty
- 3) Post: return index of item if found, otherwise -1
- 4) index $\leftarrow 0$ // Start checking from the beginning
- 5) while index < list.Count and list[index] $\neq$ item
- 6) index $\leftarrow$ index + 1 // Move to next element
- 7) end while
- 8) if index < list.Count and list[index] = item
- 9) return index // Item found, return position
- 10) end if
- 11) return -1 // Item not found
- 12) end SequentialSearch

Example:

Let's say we are searching for the number 54 in the list: [12, 33, 54, 66, 99]

Step-by-step:

index = 0 $\rightarrow$ 12 $\neq$ 54 $\rightarrow$ move on

index = 1 $\rightarrow$ 33 $\neq$ 54 $\rightarrow$ move on

index = 2 $\rightarrow$ 54 = 54 $\rightarrow$ found! return 2

Important Notes:

Does not require sorting.

Works with **any type of list** (sorted or unsorted).

Not efficient for long lists, but useful for **small or unsorted** lists.

2- Probability Search Algorithm

Probability Search is an enhanced version of the **sequential (linear) search algorithm**, designed to **improve search performance** over time in cases where **certain elements are accessed more frequently than others**.

Key Concept:

The algorithm does not just look for a target item — it **adapts** based on how often each item is searched. More frequently accessed items are moved **closer to the front** of the list, reducing the average number of comparisons needed for future searches.

How It Works:

- The algorithm searches for the target item **sequentially** (from beginning to end).
- If the item is **found**, it is **swapped with its immediate predecessor** (the item just before it).
- As a result, frequently searched items gradually **move toward the beginning** of the list.
- Over time, this reduces the average search time for commonly used items, even though the **worst-case time complexity remains $O(n)$** .

Why It's Useful:

- In real-world applications, **not all data is accessed equally**.
- For example, in an autocomplete system or a cache, some elements are accessed repeatedly.
- By prioritizing more likely items (based on usage frequency), Probability Search **optimizes user experience and efficiency**.

```

1) algorithm ProbabilitySearch(list, item)
2)   Pre: list  $\neq \emptyset$ 
3)   Post: a boolean indicating where the item is found or not;
           in the former case swap founded item with its predecessor
4)   index  $\leftarrow 0$ 
5)   while index < list.Count and list[index]  $\neq$  item
6)     index  $\leftarrow$  index + 1
7)   end while
8)   if index  $\geq$  list.Count or list[index]  $\neq$  item
9)     return false
10)  end if
11)  if index > 0
12)    Swap(list[index], list[index - 1])
13)  end if
14)  return true
15) end ProbabilitySearch

```

Step-by-Step Explanation

Lines 1–3: Initialization

- 1) algorithm ProbabilitySearch(*list*, *item*)
- 2) Pre: *list* $\neq \emptyset$
- 3) Post: a boolean indicating whether the item is found or not;
if found, swap it with its predecessor

The function receives a list and a target item to search.

The list must not be empty.

It returns true if the item is found (and modifies the list), otherwise false.

Line 4: Start Index

4) $\text{index} \leftarrow 0$

Start searching from the beginning of the list.

Lines 5–7: Sequential Search Loop

5) while $\text{index} < \text{list.Count}$ and $\text{list}[\text{index}] \neq \text{item}$

6) $\text{index} \leftarrow \text{index} + 1$

7) end while

This loop moves through the list until:

It finds the item, or

It reaches the end of the list.

Lines 8–10: Check if Item Was Found

8) if $\text{index} \geq \text{list.Count}$ or $\text{list}[\text{index}] \neq \text{item}$

9) return false

10) end if

If the loop exited and the item was **not** found:

Return false.

Lines 11–13: Swap with Predecessor

11) if $\text{index} > 0$

12) $\text{Swap}(\text{list}[\text{index}], \text{list}[\text{index} - 1])$

13) end if

If the found item is **not the first element** in the list,

- Swap it with its **previous item**.
- This is what improves efficiency: items that are searched more often will move closer to the front over time.

Line 14: Return Success

14) return true

The item was found and (possibly) swapped → return true.

Time Complexity:

- ✓ **Best case:** $O(1)$ (if the item is at the front).
- ✓ **Worst case:** $O(n)$ (still a linear scan).
- ✓ **Average case:** Improves over time if searches are non-uniform.

Advantages:

Simple to implement.

Efficient when some elements are accessed far more than others.

Adaptive behavior improves real-world performance.

Limitations:

Not suitable if all elements have equal search probability.

Not ideal when the list must maintain a fixed order.

Example: Initial List: [23, 7, 12, 99, 101]

Let's say we **search for 99**, and then **search for 12**.

First Search: Search for 99

Start scanning from the beginning:

23 ✕

7 ✕

12 ✕

99 ✓ → Found at index 3

Swap 99 with its predecessor (index 2): swap **99** and **12**

List after 1st search: [23, 7, 99, 12, 101]

Second Search: Search for 12

Start scanning:

23 ✕

7 ✕

99 ✕

12 ✓ → Found at index 3

Swap 12 with its predecessor (index 2): swap **12** and **99**

List after 2nd search: [23, 7, 12, 99, 101]

As you can see, **frequently searched items move forward** in the list to improve future search speed.

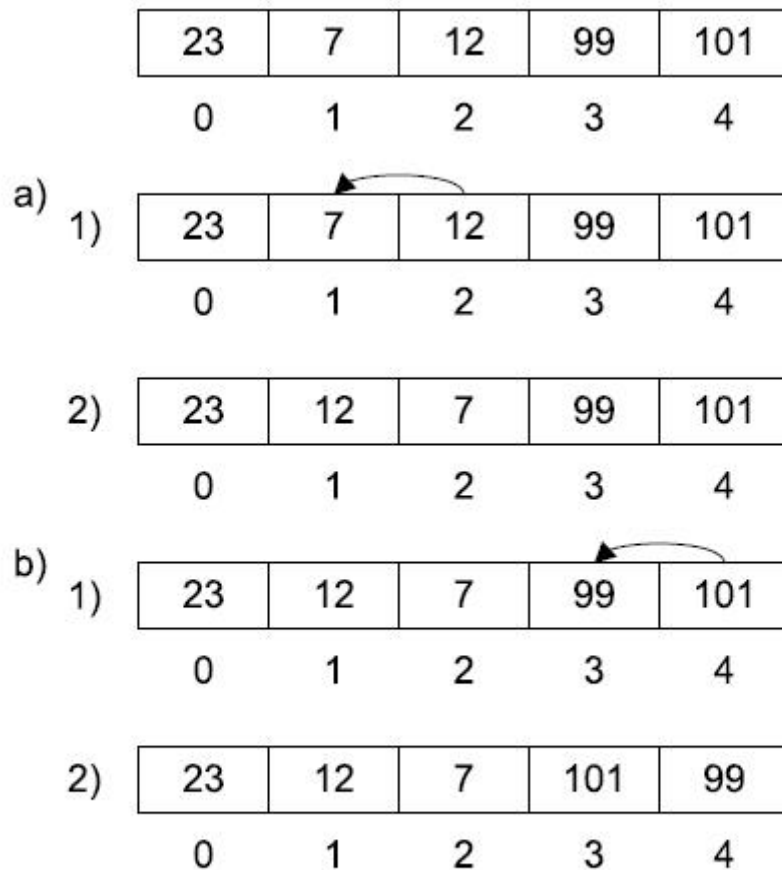


Figure 10.1: a) Search(12), b) Search(101)

3- Binary Search Algorithm

What is Binary Search?

Binary Search is an **efficient searching algorithm** used to find an element in a **sorted list or array**.

It works by **repeatedly dividing the search interval in half** and comparing the target value to the middle element.

Key Conditions:

The list **must be sorted** (ascending or descending).

It uses the **Divide and Conquer** strategy.

Time Complexity:

Best case: **O(1)** (if the item is in the middle)

Worst/Average case: **O(log n)**

Algorithm Steps (Pseudocode)

- 1) algorithm BinarySearch(list, item)
- 2) Pre: list is sorted in ascending order
- 3) Post: returns index of item if found, otherwise -1
- 4) low $\leftarrow$ 0
- 5) high $\leftarrow$ list.Count - 1
- 6) while low $\leq$ high do
- 7) mid $\leftarrow$ (low + high) / 2
- 8) if list[mid] = item then
- 9) return mid
- 10) else if list[mid] < item then
- 11) low $\leftarrow$ mid + 1
- 12) else
- 13) high $\leftarrow$ mid - 1
- 14) end if
- 15) end while
- 16) return -1
- 17) end BinarySearch

Explanation of Steps:

Step 1–3: Function takes a sorted list and the item to search for.

Step 4–5: Initialize low and high pointers to the beginning and end of the list.

Step 6: As long as low is less than or equal to high, repeat the process:

Step 7: Find the middle index of the current search interval.

Step 8–9: If the middle element matches the item → success.

Step 10–13: Otherwise, adjust the search interval:

If the item is greater than the middle → search the **right half**.

If less → search the **left half**.

Step 16: If the loop ends without a match, return -1 (item not found).

Example:

Given List:

list = [3, 7, 12, 18, 25, 39, 51]

item = 25

Table 1 Steps of Binary Search Algorithm

Step	low	high	mid	list[mid]	Compare with 25	Action
1	0	6	3	18	$18 < 25$	$\text{low} = \text{mid} + 1 = 4$
2	4	6	5	39	$39 > 25$	$\text{high} = \text{mid} - 1 = 4$
3	4	4	4	25	$25 == 25$	return 4

✓ **Output: Index of 25 is 4**

If the item was not found: Try searching for 100 in the same list:

- Steps would narrow the range and end with $\text{low} > \text{high}$.

- Output: -1 (item not found).

Questions

Question 1: Hybrid Search Strategy

You are given a large dataset that is mostly **static** (rarely changes) but needs to support **frequent and fast searches**. What searching algorithm would you choose and why? Also, what **preprocessing** (if any) would improve performance?

Question 2: Probability Search Effectiveness

Suppose you frequently search the item X in the list

[A, B, C, D, E] and every time you find X, it is swapped with its predecessor (Probability Search).

If X was initially at position 4, how many **successful searches** would it take for X to reach the **front** of the list?

Question 3: Binary Search Edge Case

You are using **Binary Search** on a **sorted list with duplicates**:

[3, 5, 7, 7, 7, 9, 11] You want to return the **first occurrence** of the number 7.

- What modification is needed to the binary search algorithm?

Question 4: Algorithm Identification

You are given the following pseudocode snippet:

```
index ← 0
```

```
while index < list.Count and list[index] ≠ item
```

```
    index ← index + 1
```

```
end while
if index < list.Count
    return index
return -1
```

- Which searching algorithm is this?
- What is its **best case** and **worst case** complexity?

Question 5: Combined Scenario

Suppose you have the following unsorted list:

[5, 2, 9, 2, 3, 9, 1]

You want to:

- Frequently search for 9 (high priority item)
- Minimize average search time
- Keep the list unsorted

Which search method would **outperform** others over time and why?

CHAPTER 6 :Divide-and-Conquer

Divide-and-Conquer is a powerful algorithm design paradigm used to solve complex problems by **breaking them down** into smaller, more manageable sub-problems. It follows a three-step process:

✓ **Divide:**

Break the original problem into **smaller sub-problems**, usually of the same type.

✓ **Conquer:**

Solve the sub-problems **recursively**. If the sub-problems are still too large, divide them again.

✓ **Combine:**

Merge the results of the sub-problems to form the final solution to the original problem.

Why Use Divide-and-Conquer?

- ✓ It helps in solving **large or complex problems** efficiently.
- ✓ It often leads to algorithms with **better time complexity**.
- ✓ It allows for **parallel processing**, since sub-problems can be solved independently.

6.1 Divide-and-Conquer Paradigm

Cases:

- ✓ **Recursive Case:** Subproblems are still large and need to be divided further.
- ✓ **Base Case:** Subproblems are small and can be solved directly.

Example: Merge Sort

- ✓ **Divide:** Split the array into two halves.
- ✓ **Conquer:** Sort each half recursively using Merge Sort.
- ✓ **Combine:** Merge the two sorted halves.

Time complexity:

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

Recurrences

A **recurrence relation** is a formula that expresses the running time of a recursive algorithm in terms of the running time on smaller inputs.

Example:

$$T(n) = 2T(n/2) + O(n)$$

This means: to solve a problem of size n , you solve two subproblems of size $n/2$ and spend $O(n)$ time to combine the results.

6.2 Methods to Solve Recurrences

1. Substitution Method

Guess the solution (e.g., $T(n) = O(n \log n)$)

Prove it using mathematical induction.

2. Recursion Tree Method

Visualize the recurrence as a tree.

Add up the work done at each level.

Compute the total cost from root to leaves.

Example:

Each level of Merge Sort does $O(n)$ work, and there are $\log n$ levels, so total cost = $O(n \log n)$.

3. Master Theorem

Used for recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

Where:

$a \geq 1$: number of subproblems

$b > 1$: size reduction factor

$f(n)$: cost of dividing and combining

Maximum-Subarray Problem: Overview

Problem:

Given an array representing **daily stock price changes**, find a **contiguous subarray** that yields the **maximum possible sum**.

This models the best interval to **buy and sell** a stock to **maximize profit**, knowing future prices.

Transformation:

Instead of using the prices directly, calculate **daily changes** in price.

Then, find a **contiguous subarray** in this changes array with the **maximum sum**.

For example:

Prices: [100, 113, 110, 85, 105, ...]

Daily changes: [+13, -3, -25, +20, -3, ...]

Now find the subarray of these changes with the **highest sum** → maximum profit.

Brute-force Approach

Try all pairs of buy/sell dates.

Time complexity: $O(n^2)$ → inefficient for large datasets.

Efficient Solution Using Divide-and-Conquer

Key Idea:

Split the array into two halves:

Find the **maximum subarray** in the **left half**

Find the **maximum subarray** in the **right half**

Find the **maximum subarray crossing the midpoint**

Then, return the **best** of these three.

Three Cases:

Max subarray lies **entirely in the left half**

Max subarray lies **entirely in the right half**

Max subarray **crosses the middle**

Recursive Algorithm (Pseudocode):

FIND-MAXIMUM-SUBARRAY(A, low, high)

1. if high == low:

 return (low, high, A[low]) // base case: one element

2. else:

 mid = floor((low + high)/2)

 (left-low, left-high, left-sum) = FIND-MAXIMUM-SUBARRAY(A, low, mid)

 (right-low, right-high, right-sum) = FIND-MAXIMUM-SUBARRAY(A, mid+1, high)

 (cross-low, cross-high, cross-sum) = FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)

 if left-sum $\geq$ right-sum and left-sum $\geq$ cross-sum:

 return (left-low, left-high, left-sum)

 elif right-sum $\geq$ left-sum and right-sum $\geq$ cross-sum:

 return (right-low, right-high, right-sum)

 else:

 return (cross-low, cross-high, cross-sum)

FIND-MAX-CROSSING-SUBARRAY

Goes **left** from the middle to find max sum ending at mid.

Goes **right** from the middle+1 to find max sum starting there.

Returns their **combined max sum** crossing the midpoint.

Time complexity: $O(n)$

Time Complexity Analysis:

Recurrence relation: $T(n) = 2T(n/2) + O(n)$

This is identical to merge sort.

Using the **Master Theorem:** $T(n) = \Theta(n \log n)$

✓ **Much faster** than brute-force $O(n^2)$

⚡ **Bonus: Even Faster?**

Yes. There's a **linear time** algorithm (Kadane's Algorithm) that solves it in $O(n)$ — without recursion or divide-and-conquer.

But this divide-and-conquer approach is excellent for teaching **recursive problem solving**.

Strassen's Algorithm for Matrix Multiplication (Divide-and-Conquer)

Problem : Multiply two $n \times n$ matrices A and B to get a matrix C.

Traditional Approach

The standard method performs:

N^2 entries to compute.

Each entry takes n multiplications and additions.

Total time: $\Theta(n^3)$.

Strassen's Insight

Reduce the number of **recursive multiplications** in the divide-and-conquer approach.

Instead of 8 recursive multiplications in the naive recursive approach, Strassen uses only **7 recursive matrix multiplications**, along with additional matrix additions/subtractions.

Matrix Partitioning

Assume n is a power of 2, and split each matrix:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Strassen's 7 Products

Define:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

Then compute the resulting matrix:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

Time Complexity

Each recursive step:

7 recursive calls on $n/2 \times n/2$ matrices.

Matrix addition/subtraction: $\Theta(n^2)$

So the recurrence is: $T(n) = 7T(n/2) + \Theta(n^2)$

Using the **Master Theorem**:

- $a = 7, b = 2, f(n) = n^2$
- $\log_b a = \log_2 7 \approx 2.81$

Final time complexity: $\Theta(n^{2.81})$

This is **faster than the standard** $\Theta(n^3)$ for large n .

1. Substitution Method for Solving Recurrences

The **substitution method** is a powerful technique used to solve recurrences, especially when analyzing the time complexity of divide-and-conquer algorithms.

Steps of the Substitution Method

Guess the Form of the Solution

Based on the recurrence, estimate the asymptotic behavior (e.g., $T(n) = O(n \log n)$ or $T(n) = O(n^2)$).

Prove the Guess Using Mathematical Induction

Use **induction** to show that your guessed function bounds the recurrence from above (or below), often by assuming it holds for smaller inputs.

Example

Given the recurrence: $T(n) = 2T(\lfloor n/2 \rfloor) + n$

This recurrence commonly appears in **merge sort**. Let's solve it using substitution.

Step 1: Guess the Solution

Guess: $T(n) = O(n \log n)$

Assume: $T(n) \leq c \cdot n \cdot \log n$, for some constant $c > 0$

Step 2: Use Induction to Prove the Guess

Assume the inequality holds for values smaller than n , especially $\lfloor n/2 \rfloor$:

$$T(\lfloor n/2 \rfloor) \leq c \cdot (n/2) \cdot \log(n/2)$$

Now plug this into the recurrence:

$$\begin{aligned} T(n) &= 2T(\lfloor n/2 \rfloor) + n \\ &\leq 2[c \cdot (n/2) \cdot \log(n/2)] + n \\ &= c \cdot n \cdot \log(n/2) + n \\ &= c \cdot n \cdot (\log n - \log 2) + n \\ &= c \cdot n \cdot \log n - c \cdot n + n \end{aligned}$$

So,

$$\begin{aligned} T(n) &\leq c \cdot n \cdot \log n - c \cdot n + n \\ T(n) &\leq c \cdot n \cdot \log n, \text{ as long as } c \geq 1 \end{aligned}$$

Thus, the guess holds.

Handling Base Cases

You must **check the base case**, e.g., $T(1) = 1$. For $n = 1$, $T(1) \leq c \cdot 1 \cdot \log 1 = 0$, which is false.

Solution: Use a *threshold* $n_0 > 1$, and only prove the inequality for $n \geq n_0$. You can separately verify the small values of $T(n)$.

Tips for Guessing the Solution

Use recursion trees or similar known recurrences.

Try changing variables if the recurrence is nonstandard.

If the math doesn't work, try strengthening the induction hypothesis (e.g., subtracting a lower-order term like $T(n) \leq cn - d$).

Key Takeaways

The substitution method is about **guessing** and **proving**.

It's useful when **Master Theorem** doesn't apply or when you're looking for deeper insight.

Don't forget to **check base cases** and **use strong induction** if needed.

Example: Solve the recurrence:

$$T(n) = 2T(n/2) + n, \text{ with base case } T(1) = 1$$

Step 1: Guess the form of the solution

We guess: $T(n) = O(n \log n)$

So let's try to **prove**: $T(n) \leq c \cdot n \cdot \log n$ for some constant $c > 0$

Step 2: Inductive Hypothesis

Assume this inequality holds for all values smaller than n , especially for $n/2$:

$$T(n/2) \leq c \cdot (n/2) \cdot \log(n/2)$$

✓ Step 3: Plug into the recurrence

Start from: $T(n) = 2T(n/2) + n$

Now substitute using the inductive hypothesis:

$$\begin{aligned}
T(n) &\leq 2[c \cdot (n/2) \cdot \log(n/2)] + n \\
&= c \cdot n \cdot \log(n/2) + n \\
&= c \cdot n \cdot (\log n - \log 2) + n \\
&= c \cdot n \cdot \log n - c \cdot n + n
\end{aligned}$$

Group terms: $T(n) \leq c \cdot n \cdot \log n - c \cdot n + n = c \cdot n \cdot \log n - (c \cdot n - n)$

To ensure: $T(n) \leq c \cdot n \cdot \log n$

we need: $(c \cdot n - n) \geq 0 \rightarrow c \geq 1$

Step 4: Check the base case

Try $T(1)$:

Left-hand side: $T(1) = 1$

Right-hand side: $c \cdot 1 \cdot \log(1) = 0$

So $1 \leq 0$ ✗ $\rightarrow$ This fails.

But we can fix it by **choosing c large enough and starting induction at $n \geq 2$** .

So the final conclusion: $T(n) = O(n \log n)$

2. Recursion Tree Method

The **Recursion Tree Method** is a visual technique used to **solve recurrence relations**—especially those arising in **divide-and-conquer algorithms**.

It helps estimate the **total time complexity** by breaking down the recursive calls into a tree, calculating the **work done at each level**, and then **summing up all levels**.

Main Steps

Draw the recursion tree:

Each node represents a subproblem and the cost (work) of solving it.

The root node is the original problem $T(n)$.

Each child node represents recursive calls.

Calculate the cost per level:

Determine how many nodes are at each level and the cost at each node.

Count the number of levels:

How many times the problem size can be divided until it hits the base case.

Sum the total cost:

Add up the work done at each level to estimate the total time.

(Optional) Use the **substitution method** to verify your guess.

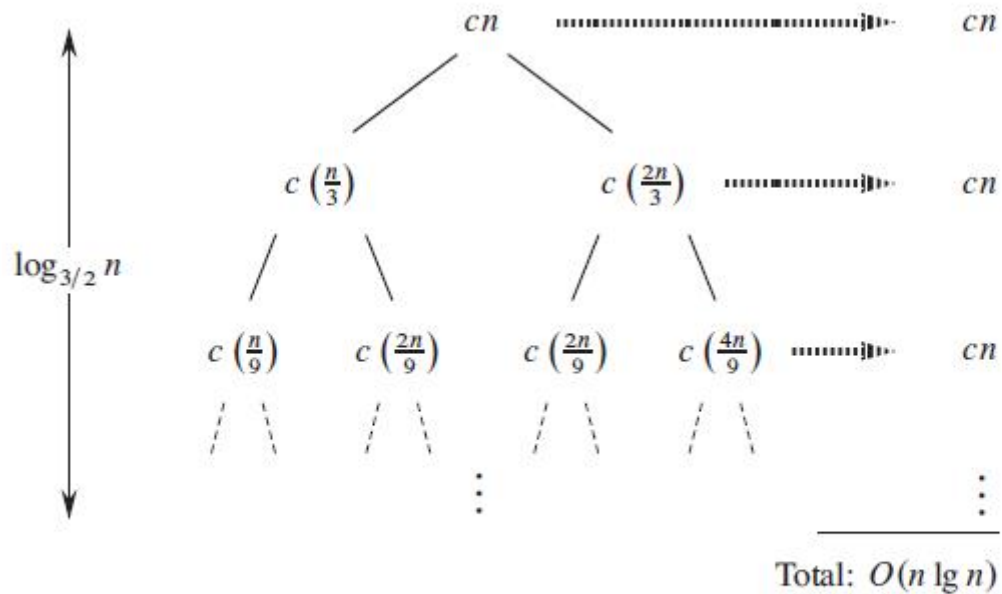


Figure 4.6 A recursion tree for the recurrence $T(n) = T(n/3) + T(2n/3) + cn$.

Example

Let's solve this recurrence:

$$T(n) = 2T(n/2) + n$$

Step 1: Draw the recursion tree

At each level:

There are 2^i subproblems of size $n/2^i$

Each node does work = $n/2^i$

Table 1 Recursion Tree Method

Level	of Nodes	Work per Node	Total Work
0	1	n	n
1	2	n/2	n
2	4	n/4	n
...	...	...	...
$\log_2 n$	n	1	n

Each level costs n and there are $\log_2 n + 1$ levels.

Step 2: Sum the work across all levels

$$T(n) = n + n + n + \dots + n = n \log n + n = \Theta(n \log n)$$

✓ Final Result

The time complexity of the recurrence

$$T(n) = 2T(n/2) + n$$

$$T(n) = \Theta(n \log n)$$

3. The Master Method for Solving Recurrences

The **Master Method** provides a direct way to find asymptotic bounds for recurrence relations of the form: $T(n) = aT(n/b) + f(n)$

Where:

$a \geq 1$: number of recursive subproblems,

$b > 1$: the factor by which the problem size is divided,

$f(n)$: the cost of work done outside the recursive calls (e.g., dividing and combining subproblems).

Master Theorem Cases

Let $T(n) = aT(n/b) + f(n)$, and compare $f(n)$ to $n^{\log_b a}$:

Case 1: Polynomially Smaller

If:

$$f(n) = O(n^{\log_b a - \varepsilon}) \quad \text{for some } \varepsilon > 0$$

Then:

$$T(n) = \Theta(n^{\log_b a})$$

✓ Recursion dominates.

Case 2: Same Order (Balanced)

If:

$$f(n) = \Theta(n^{\log_b a} \cdot \log^k n) \quad \text{for some } k \geq 0$$

Then:

$$T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$$

✓ Both recursion and combine cost equally contribute.

Case 3: Polynomially Larger

If:

$$f(n) = \Omega(n^{\log_b a + \varepsilon}) \quad \text{for some } \varepsilon > 0$$

And if the following **regularity condition** holds:

$$a \cdot f(n/b) \leq c \cdot f(n) \quad \text{for some } c < 1 \text{ and large } n$$

Then:

$$T(n) = \Theta(f(n))$$

✓ The cost outside recursion dominates.

Example 1:

$$T(n) = 2T(n/2) + n$$

- $a = 2, b = 2, f(n) = n$
- Compare: $f(n) = n, n^{\log_2 2} = n$
- $\rightarrow$ Case 2 applies

✓ Answer: $T(n) = \Theta(n \log n)$

Example 2:

$$T(n) = 4T(n/2) + n$$

- $a = 4, b = 2$, so $n^{\log_2 4} = n^2$
- $f(n) = n$, which is smaller than n^2

$\rightarrow$ Case 1 applies

✓ Answer: $T(n) = \Theta(n^2)$

Example 3:

$$T(n) = 2T(n/2) + n^2$$

- $a = 2, b = 2$, so $n^{\log_2 2} = n$
- $f(n) = n^2$, which is larger than n

→ Try Case 3:

- Regularity condition: $2 \cdot (n/2)^2 = n^2/2 \leq c \cdot n^2$ for $c = 0.9$

✓ Holds. So:

✓ Answer: $T(n) = \Theta(n^2)$

Table 2. Summary Master Theorem Cases

Case	Condition on $f(n)$	Result
1	$f(n) = O(n^{\log_b a - \varepsilon})$	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$	$T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$
3	$f(n) = \Omega(n^{\log_b a + \varepsilon})$, and regularity	$T(n) = \Theta(f(n))$

QUESTIONS

Q1: Use the substitution method to find an upper bound for the recurrence:

$T(n) = 2T(n/2) + n$, and show that $T(n) = O(n \log n)$.

Q2: Use the substitution method to prove that the recurrence

$T(n) = T(n - 1) + n$ has solution $T(n) = O(n^2)$.

Q3: Use a recursion tree to determine a good asymptotic bound on the recurrence:

$T(n) = 3T(n/4) + n$. Then verify your answer using the substitution method.

Q4: Draw the recursion tree for: $T(n) = 2T(n/2 + 5) + n$, and find an upper bound for $T(n)$.

Q5: Use a recursion tree to analyze $T(n) = 4T(n/2 + 2) + n$. What's your guess for the upper bound?

Q6: Use the master method to solve: $T(n) = 2T(n/2) + n$.

Q7: Solve the recurrence using the Master Theorem: $T(n) = 4T(n/2) + n^2$.

Q8: Use the master method to find the complexity of: $T(n) = T(n/2) + \Theta(1)$. Show that the answer is $T(n) = \Theta(\log n)$.

Q9: Solve using the master method: $T(n) = 8T(n/2) + n^3$

CHAPTER 7 : Design Strategies

In the study of algorithms, **design strategies** refer to the general approaches or techniques used to **construct efficient algorithms** for solving computational problems. Rather than starting from scratch for each new problem, computer scientists rely on a set of well-established design paradigms that guide the algorithm development process.

This chapter introduces some of the most important algorithm design strategies, including:

- ✓ **Divide and Conquer**
- ✓ **Greedy Method**
- ✓ **Dynamic Programming**
- ✓ **Backtracking**
- ✓ **Randomized Algorithms**

Each strategy has its own principles, strengths, and typical applications. The choice of a suitable strategy depends on the nature of the problem, the constraints, and the desired performance.

Understanding these strategies is essential because:

- ✓ They offer **reusable solutions** and patterns across different problem domains.
- ✓ They help **analyze time and space complexity** more effectively.
- ✓ They enable the creation of **optimized and scalable algorithms**.

By the end of this chapter, students should be able to:

- ✓ Recognize which design strategy is appropriate for a given problem.
- ✓ Apply the strategy to develop an efficient algorithm.
- ✓ Compare and contrast different strategies in terms of their use cases and performance.

7.1 Greedy Algorithms

- Greedy algorithms are a problem-solving approach that makes locally optimal choices at each step to achieve a globally optimal solution.
- Core principle: "Choose the best immediate option without reconsidering future consequences."
- Applications: Optimization problems (e.g., scheduling, shortest path, resource allocation) where greedy choices lead to an optimal solution.

1. Key Properties of Greedy-Solvable Problems

1. Greedy Choice Property:

- A globally optimal solution can be reached by selecting locally optimal choices at each step.
- Example: In coin change problems, picking the largest coin first minimizes the total coins used.

2. Optimal Substructure:

- An optimal solution to the problem contains optimal solutions to its subproblems.
- Example: In the shortest path problem, subpaths of the shortest path are also the shortest.

2. Steps to Design a Greedy Algorithm

1. Problem Formulation:

- Define the problem as a set of choices (e.g., coins, jobs, edges in a graph).

2. Greedy Selection Criterion:

- Identify the rule for selecting the "best" option at each step (e.g., largest value, smallest weight).

3. Feasibility Check:

- Ensure the selected choice contributes to a valid solution.

4. Iterate Until Completion:

- Repeat selection until all subproblems are solved.

5. Prove Optimality:

- Mathematically verify that the greedy approach yields a global optimum.

3. Classic Examples

1. Coin Change Problem

- Goal: Make change for an amount using the fewest coins.
- Greedy Rule: Always pick the largest coin $\leq$ remaining amount.
- Caveat: Works only for specific coin systems (e.g., USD). Fails for arbitrary systems (e.g., coins = [1, 3, 4] for amount 6).

Pseudocode:

```
def coin_change(coins, amount):
    coins.sort(reverse=True) # Sort descending
    count = 0
    for coin in coins:
        while amount >= coin:
            amount -= coin
            count += 1
    return count if amount == 0 else -1
```

7.2 Fractional Knapsack

- Goal: Maximize value in a knapsack with limited capacity (items can be split).
- Greedy Rule: Pick items with the highest value-to-weight ratio first.

Pseudocode:

```
def fractional_knapsack(items, capacity):
    items.sort(key=lambda x: x.value/x.weight, reverse=True) # Sort by value/weight
    total_value = 0
    for item in items:
        if capacity >= item.weight:
            capacity -= item.weight
            total_value += item.value
        else:
            fraction = capacity / item.weight
            total_value += fraction * item.value
            break
    return total_value
```

3. Activity Selection

- Goal: Schedule maximum non-overlapping activities.
- Greedy Rule: Always pick the activity with the earliest end time.

Pseudocode:

```
def activity_selection(activities):
    activities.sort(key=lambda x: x.end_time)  # Sort by end time
    selected = [activities[0]]
    last_end = activities[0].end_time
    for activity in activities[1:]:
        if activity.start_time >= last_end:
            selected.append(activity)
            last_end = activity.end_time
    return selected
```

4. When Greedy Fails

- Counterexample: 0/1 Knapsack
- Problem: Fill a knapsack with indivisible items (take or leave).
- ✓ Greedy failure: Selecting items by value/weight ratio isn't always optimal.
- ✓ Items: [(value=60, weight=10), (value=100, weight=20), (value=120, weight=30)]
- ✓ Capacity: 50
- ✓ Greedy: $60 + 100 = 160$ ($10+20=30$ weight).
- ✓ Optimal: $100 + 120 = 220$ ($20+30=50$ weight).

5. Proof Techniques

Greedy Stays Ahead:

Show that after each step, the greedy solution is at least as good as any other partial solution.

Used in Activity Selection.

Exchange Argument:

Prove that any optimal solution can be transformed into the greedy solution without worsening it.

Used in Fractional Knapsack.

6. Advantages & Limitations

Advantages	Limitations
Simple and intuitive.	Not universally applicable.
Efficient (often $O(n \log n)$).	Requires proof of optimality.
Low memory footprint.	May fail for subtle problem variants.

7. Practice Problems

1. Huffman Coding: Compress data using variable-length codes (greedily merge least frequent nodes).
2. Dijkstra's Algorithm: Find shortest paths in graphs (greedily pick closest unvisited node).
3. Minimum Spanning Tree (Prim/Kruskal): Connect nodes with least total edge weight.

7.3 Dynamic Programming (DP)

1. Introduction

- ✓ Dynamic Programming is a method for solving complex problems by breaking them into smaller subproblems, solving each once, and storing their solutions (memoization/tabulation).

- ✓ Core Idea: Avoid recomputing the same subproblems repeatedly—store results and reuse them.
- ✓ Best For: Optimization problems with overlapping subproblems and optimal substructure.

2. Key Properties of DP Problems

1. Overlapping Subproblems

- The same subproblems recur multiple times.
- Example: In Fibonacci, `fib(5)` requires `fib(4)` and `fib(3)`, both of which reuse `fib(2)`.

2. Optimal Substructure

- An optimal solution to the main problem can be built from optimal solutions to its subproblems.
- Example: The shortest path between two cities via intermediate cities requires the shortest subpaths.

3. DP Approaches

1. Top-Down (Memoization)

- Recursive with caching. Solve the main problem recursively, storing solutions to subproblems.
- Advantage: Intuitive (like recursion).
- Disadvantage: Stack overflow for deep recursion.

2. Bottom-Up (Tabulation)

- Iterative with a table. Solve subproblems first, then combine to solve the main problem.

- Advantage: No recursion overhead; efficient memory use.

- Disadvantage: Requires defining order of computation.

4. Steps to Solve DP Problems

1. Characterize the Structure

- Identify subproblems and state transitions.

2. Define Recurrence Relation

- Write a formula for the solution using subproblems.

3. Choose DP Approach

- Decide between top-down or bottom-up.

4. Implement

- Add memoization (cache) or tabulation (table).

5. Reconstruct Solution

- Backtrack from the final state to get choices (e.g., selected items in knapsack).

5. Classic Examples

1. Fibonacci Sequence

- Recurrence: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

- Top-Down (Python):

```

1) function Fibonacci(n, memo)
2)   if n ≤ 1
3)     return n
4)   if memo[n] ≠ undefined
5)     return memo[n]
6)   memo[n] ← Fibonacci(n - 1, memo) + Fibonacci(n - 2, memo)
7)   return memo[n]

```

- Bottom-Up (Python):

```

FUNCTION fib(n):
  IF n ≤ 1:
    RETURN n

  dp[0] = 0
  dp[1] = 1

  FOR i FROM 2 TO n:
    dp[i] = dp[i-1] + dp[i-2]

  RETURN dp[n]
END FUNCTION

```

2. 0/1 Knapsack Problem

Given:

A set of n items, each with:

weight[i]: weight of item i

value[i]: value of item i

A knapsack of maximum capacity W

Goal:

Select a subset of items such that:

The total weight is $\leq W$

The **total value is maximized**

Each item is **either taken or not** (0/1 – no fractions)

Pseudocode (Bottom-Up Dynamic Programming):

```
1) function Knapsack01(weights, values, W, n)
2)   Create table dp[0...n][0...W] initialized to 0
3)
4)   for i from 1 to n:
5)     for w from 1 to W:
6)       if weights[i-1] ≤ w:
7)         include = values[i-1] + dp[i-1][w - weights[i-1]]
8)         exclude = dp[i-1][w]
9)         dp[i][w] = max(include, exclude)
10)      else:
11)        dp[i][w] = dp[i-1][w]
12)   return dp[n][W]
```

Example:

values = [60, 100, 120]

weights = [10, 20, 30]

W = 50

n = 3

Result:

Maximum value = **220**

Pick items with weights 20 and 30

✓ Key Concepts:

Bottom-up fills a table of size $n+1$ by $W+1$

$dp[i][w]$ stores the maximum value for the first i items and capacity w

Time Complexity: **$O(nW)$**

Space Complexity: **$O(nW)$** (can be optimized to **$O(W)$** with 1D array)

3. Longest Common Subsequence (LCS)

Given two sequences:

$X[0..m-1]$

$Y[0..n-1]$

Find the length of the longest subsequence common to both sequences.

A **subsequence** is a sequence that appears in the same relative order, but not necessarily contiguous.

Pseudocode (Bottom-Up DP):

- 1) function LCS(X, Y, m, n):
- 2) Create table $dp[0..m][0..n]$ initialized to 0
- 3)
- 4) for i from 1 to m :
- 5) for j from 1 to n :
- 6) if $X[i-1] == Y[j-1]$:
- 7) $dp[i][j] = dp[i-1][j-1] + 1$
- 8) else:
- 9) $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$
- 10)

11) return dp[m][n]

Example:

X = "AGGTAB"

Y = "GXTXAYB"

DP Table Result:

The length of LCS is **4**, and the LCS is "**GTAB**"

Key Concepts:

dp[i][j] stores the length of the LCS of X[0..i-1] and Y[0..j-1]

When characters match: add 1 to diagonal value

When not: take max from left or top cell

Time Complexity: **$O(m \times n)$**

Space Complexity: **$O(m \times n)$** (can be optimized to **$O(\min(m,n))$**)

7. DP vs. Greedy vs. Divide-and-Conquer

Feature	Dynamic Programming	Greedy Algorithm	Divide and Conquer
Subproblem Overlap	Yes	No	No
Reuse Subsolutions	Yes (via memoization)	No	No
Optimal Substructure	Required	Required	Required
Approach	Bottom-up or Top-down	Step-by-step optimal choice	Divide, solve, combine
Time Complexity	Higher than Greedy	Often fastest	Logarithmic to polynomial
Memory Use	High (stores results)	Low	Medium
Examples	LCS, Knapsack	Activity Selection, Kruskal	Merge Sort, Binary Search

Questions

Dynamic Programming

Q1. Given an array `arr[]` of n integers, write a bottom-up dynamic programming solution to find the length of the **Longest Increasing Subsequence (LIS)**.

- Show the DP array construction and explain your transitions.

Q2. Implement a **top-down memoized recursive algorithm** for the 0/1 Knapsack Problem.

- Explain how memoization helps in reducing time complexity.
- Apply your algorithm to:
values = [60, 100, 120], weights = [10, 20, 30], W = 50.

Q3. Write a dynamic programming algorithm to solve the **Longest Common Subsequence (LCS)** of two strings.

- Apply your algorithm to: X = "AGGTAB", Y = "GXTXAYB"
- Show the complete DP table and the final sequence.

Greedy Algorithms

Q4. Prove that the **greedy algorithm** for the Activity Selection Problem (selecting maximum non-overlapping activities) always gives an optimal solution.

- Justify your answer with an example and a proof sketch.

Q5. You are given coin denominations {1, 3, 4}. For a given amount $n = 6$, use:

- a) Greedy strategy (choose largest coin first),
- b) Dynamic programming.

- Show both solutions and explain why the greedy solution fails.

Q6. Design a greedy algorithm to find a **minimum spanning tree** using **Kruskal's algorithm**.

- Apply it step-by-step to a weighted graph of your choice with at least 6 nodes.
- Show union-find operations and edge selection.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
2. Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
3. Kleinberg, J., & Tardos, É. (2005). Algorithm Design. Pearson.
4. Dasgupta, S., Papadimitriou, C. H., & Vazirani, U. V. (2008). Algorithms. McGraw-Hill.
5. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.
6. Skiena, S. S. (2020). The Algorithm Design Manual (3rd ed.). Springer.
7. Brassard, G., & Bratley, P. (1996). Fundamentals of Algorithmics. Prentice Hall.
8. Knuth, D. E. (1997). The Art of Computer Programming, Volume 1: Fundamental Algorithms (3rd ed.). Addison-Wesley.
9. Manber, U. (1989). Introduction to Algorithms: A Creative Approach. Addison-Wesley.
10. CareerMonk. (n.d.). Algorithm Fundamentals. Retrieved from www.careermonk.com

Name Student :

1. ATM Logic

Describe how an ATM machine decides the number of bills to dispense for a given amount using a greedy algorithm approach. Write the logic in pseudocode.

2. Cash Register System

Design an algorithm to simulate a cashier giving change using the fewest coins. Assume coin denominations are [1, 5, 10, 25, 50, 100].

3. **Netflix Recommendation System**

Suggest how an algorithm could be used to recommend movies to users. Briefly outline the steps or data used.

4. **Binary Search Enhancement**

Modify the binary search algorithm to return the first and last occurrence of a target number in a sorted list with duplicates.

5. Sorting for E-Commerce

You're building a product listing page that displays items sorted by price. Which sorting algorithm would you choose and why? Consider efficiency and stability.

6. Traffic Route Optimization

GPS uses algorithms to suggest the shortest route. Explain which algorithmic strategy (greedy, divide-and-conquer, etc.) is best suited for this task and why.

Multiple-Choice Questions (MCQs)

1- What is the primary purpose of an algorithm?

- a) To write code in a specific programming language
- b) To solve a problem through a step-by-step procedure
- c) To compile software efficiently
- d) To create databases

2- Which of the following is NOT a characteristic of a valid algorithm?

- a) Finiteness
- b) Definiteness
- c) Ambiguity
- d) Input

3- Pseudocode is preferred over actual code in algorithm design because:

- a) It is faster to execute
- b) It abstracts implementation details
- c) It runs on any computer
- d) It doesn't require variables

4- Which of the following algorithm representations is best for visualization?

- a) Pseudocode
- b) Natural language
- c) Flowchart
- d) Machine code

5- An algorithm that solves problems by dividing them into sub-problems uses which strategy?

- a) Greedy
- b) Divide and Conquer
- c) Brute Force
- d) Backtracking

6- The time complexity of Binary Search is:

- a) $O(n)$

- b) $O(n^2)$
- c) $O(\log n)$
- d) $O(n \log n)$

7- Which notation provides the tightest bound on the running time of an algorithm?

- a) Big-O
- b) Big-Omega
- c) Big-Theta
- d) Little-o

8- In Bubble Sort, the best-case time complexity is achieved when:

- a) The list is already sorted
- b) The list is in reverse order
- c) All elements are the same
- d) It's applied on strings

9- Which algorithm is an example of a greedy algorithm?

- a) Binary Search
- b) Activity Selection
- c) Merge Sort
- d) Quick Sort

10- What is the worst-case time complexity of Quick Sort?

- a) $O(n)$
- b) $O(n \log n)$
- c) $O(n^2)$
- d) $O(\log n)$

11- Which search algorithm adapts based on search frequency of elements?

- a) Binary Search
- b) Probability Search
- c) Depth-First Search
- d) Sequential Search

12- In a recursive algorithm, what is the purpose of the base case?

- a) To stop further recursive calls
- b) To double the execution time
- c) To initialize variables
- d) To handle errors

13- What does the expression $|A[i] - A[j]|$ represent in an algorithm?

- a) Absolute difference between two values
- b) Square root of sum
- c) Sorting priority
- d) GCD of values

14- Dynamic Programming is mainly used when:

- a) There is no optimal substructure
- b) Subproblems overlap and reuse is possible
- c) Problems are simple
- d) Data is random

15- Which algorithm has the time complexity $T(n) = 2T(n/2) + n$?

- a) Binary Search
- b) Merge Sort
- c) Bubble Sort
- d) Linear Search